

Automated Detection of Satellite Trails in Ground-Based Observations Using U-Net and the Probabilistic Hough Transform

A REPORT PRESENTED BY

BARON GRACIAS

Departments

Department of Physics (Cavendish Laboratory)
Institute of Astronomy

Degree

MPhil Data Intensive Science

Supervisor

Dr Eduardo Gonzalez-Solares



CLARE COLLEGE
UNIVERSITY OF CAMBRIDGE

July 2026

Abstract

Automated Detection of Satellite Trails in Ground-Based Observations Using U-Net and the Probabilistic Hough Transform

Baron Gracias

The proliferation of low-Earth-orbit satellite constellations has made automated detection and masking of satellite trails a routine requirement for wide-field observatories. This dissertation replicates the U-Net plus probabilistic Hough transform pipeline of Stoppa et al. [27] on a labelled MeerLICHT subset. The PyTorch implementation uses an architecture-faithful U-Net, a combined binary cross-entropy and squared-Dice loss, validation-only model and threshold selection, and Hough post-processing of the probability map. The replication is then examined through four analyses: residual error, model variants, training-protocol sensitivity, and inference-time strategies. The replication reproduces the paper’s recall (five-seed test recall 0.923 against 0.94) and its Hough completeness (trail-pixel recall $0.918 \rightarrow 0.990$; a pixel-level false-negative rate $0.082 \rightarrow 0.0105$ comparable to the paper’s $6.98\% \rightarrow 3.38\%$), while strict pixel precision sits below the paper and is prevalence-dependent. The residual-error analysis narrows the gap: 82.5% of false positives fall in trail-bearing patches, and separate tolerance and distance diagnostics place much of the residual within one pixel of the mask. A blinded single-author re-annotation of 64 sealed crops finds no systematic width bias between the original masks, the re-annotation, and the model. Model, training, and inference variants move the residual only slightly. The evidence therefore supports a narrow conclusion: recall and Hough completeness reproduce, while the remaining strict-precision gap is most consistent with boundary-scale annotation disagreement on this dataset.

Contents

1	Introduction and Context	5
1.1	The target pipeline	5
1.2	Background and related work	6
1.2.1	Classical trail detection	6
1.2.2	Deep learning for astronomical segmentation	6
1.2.3	The target pipeline: Stoppa et al. (2024)	6
1.3	Aims, scope, and contributions	7
1.4	Code and data availability	7
2	Dataset and Annotation Characteristics	8
2.1	The MeerLICHT subset	8
2.2	Exploratory analysis	8
2.3	Patch extraction and class imbalance	10
2.4	Dataset splits	10
2.5	Mask quality	11
3	Methodological Framing	12
3.1	Problem formulation	12
3.2	Learned segmentation with geometric post-processing	12
3.3	U-Net architecture choice	13
3.4	Combined BCE and Dice loss	13
3.5	Evaluation protocol and assumptions	13
4	Model Implementation and Training Protocol	14
4.1	Data pipeline	14
4.1.1	Data augmentation	14
4.2	Network architecture	15
4.3	Loss function	16
4.4	Hyperparameter search	16
4.5	Threshold selection	17
4.6	Hough post-processing	17

4.7	Patch CNN classifier for the two-stage variant	17
4.8	Evaluation metrics	18
4.9	Determinism and reproducibility	18
4.10	Reproducibility and software structure	18
5	Replication and Error Diagnosis	19
5.1	Replication of the target detector	19
5.2	Validation-only model selection	19
5.3	Hough improves pixel completeness	21
5.4	Uncertainty quantification	22
5.5	Qualitative prediction checks	22
5.6	Diagnostics localise much of the precision gap near boundaries	24
5.6.1	False-positive decomposition	24
5.6.2	Boundary-tolerant evaluation	24
5.6.3	False-positive distance to mask	25
5.6.4	Relative display-contrast diagnostics	26
5.6.5	Topology-aware comparison (clDice)	26
5.6.6	Blinded re-annotation audit	27
5.7	Model variants as diagnostic checks	28
5.7.1	Classifier-gated detector	28
5.7.2	Attention-gated U-Net	28
5.8	Training and inference sensitivity	30
5.8.1	Data efficiency	30
5.8.2	Hard-negative mining	30
5.8.3	Dilated soft-label pilot	31
5.8.4	Ensembling with test-time augmentation	31
5.8.5	Probability-stratified Hough check	31
6	Discussion	32
6.1	Deviations from the target paper	32
6.2	Interpreting the headline comparison	32
6.3	Limitations	33
6.4	Scientific implications	34
7	Conclusions and Future Work	36
7.1	Conclusions	36
7.2	Future work	37
	Bibliography	38

A	Hyperparameter Search Space	42
B	Supplementary Results	43
B.1	Post-hoc Trail and Error Diagnostics	43
B.2	Prediction Galleries	48
C	Use of Auto-generation Tools	50

Chapter 1

Introduction and Context

The low-Earth-orbit satellite population is growing quickly. CelesTrak listed 15,848 active satellites on 27 May 2026, of which approximately 14,600 occupied orbits below 2,000 km, including more than 10,000 Starlink spacecraft [6]. Regulatory approvals extend further: 15,000 second-generation Starlink satellites are approved [9], and policy reviews treat proposals exceeding 100,000 spacecraft as realistic [33, 34]. Each sunlit satellite crossing a wide-field exposure leaves a bright, near-linear trail that corrupts photometry and astrometry along its path and injects spurious candidates into transient-detection pipelines [10, 30].

The contamination is already measurable. Starlink trails affected under 0.5 % of Zwicky Transient Facility twilight images in late 2019 but 18 % by August 2021 [19]; Rubin Observatory simulations indicate that approximately 10 % of LSST images would contain at least one trail under a 40,000-satellite scenario [31]. Other wide-field twilight exposures can reach 30–40 % contamination [2, 10], and 4.3 ± 0.4 % of Hubble ACS/WFC images taken in 2018–2021 contained trails [15]. At these rates manual flagging does not scale: a survey instrument such as MeerLICHT [4] produces $10,560 \times 10,560$ -pixel frames throughout the night, so identifying and masking affected pixels must be automated, reliable at the pixel level, and cheap enough to run inside the reduction pipeline.

1.1 The target pipeline

Stoppa et al. [27] present ASTA, the pipeline replicated in this dissertation: a U-Net [23] trained on manually annotated MeerLICHT images, followed by a probabilistic Hough transform [13, 16] applied to the network’s output probability map. The two stages serve complementary roles. The U-Net supplies per-pixel soft detection that survives the crowded stellar background. The Hough stage exploits the one prior the network does not encode explicitly, that a satellite trail is globally linear, to connect fragmented detections and extend partially recovered trails. The published pipeline reaches precision and recall both

approaching 0.94 and was applied at scale to around 200,000 full MeerLICHT fields. The published behaviour is summarised in Section 1.2.3; the replication results are reported in Chapter 5.

1.2 Background and related work

1.2.1 Classical trail detection

Classical satellite-trail detection usually treats trails as geometric line features. Hough and Radon-family methods map image pixels into line-parameter space, making them natural tools for long, straight artefacts [8, 13, 16]. Astronomical uses include survey cleaning, masked-star validation in crowded fields, faint-streak matched filtering, space-debris and DECam workflows, and Hubble ACS trail recovery [3, 20, 24, 26, 28, 32]. Their shared limitation is sensitivity to faint, fragmented, or background-confused trails, where reliable voting and instrument-specific preprocessing become difficult; this motivates learned segmentation before geometric post-processing.

1.2.2 Deep learning for astronomical segmentation

The U-Net architecture [23], an encoder–decoder with skip connections that preserve spatial resolution, was designed for biomedical segmentation and is now common in astronomical imaging. Close precedents for thin, sparse artefacts are deepCR [37] and Cosmic-CoNN [35], both cosmic-ray segmentation systems. MaxiMask [22] covers contaminant masking including satellite and aircraft trails, and trails have since been targeted with U-Net plus line-segment detection [7], multi-band attention U-Nets [36], and end-to-end streak cataloguing [5]. No published trail-detection pipeline was found that gates a segmentation U-Net with a lightweight patch classifier, although classifiers are already used elsewhere in the MeerLICHT ecosystem [12]. Section 5.7.1 therefore treats this as a narrow follow-up test.

1.2.3 The target pipeline: Stoppa et al. (2024)

Stoppa et al. [27] trained a U-Net on 528×528 MeerLICHT patches with a combined binary cross-entropy and Dice loss, and applied a probabilistic Hough transform to the U-Net probability map as a post-processing step. On a 20,000-patch test set at threshold 0.58 the pipeline reported precision and recall both approaching 0.94, with the Hough step reducing the pixel-level false-negative rate from 6.98 % to 3.38 %. The paper does not report the training hyperparameters, making faithful replication a non-trivial search (Chapter 4).

1.3 Aims, scope, and contributions

This dissertation has three aims. The first is a faithful PyTorch replication of the Stoppa et al. [27] pipeline on the labelled MeerLICHT subset, with the network architecture reconstructed from the released Keras `model-best.h5` file, not the paper’s text alone, and with model and threshold selection locked on validation data before any test metric is inspected. The second is to diagnose the residual error of that replication: to establish where the strict precision gap against the published figures comes from, rather than to report it as a single number. The third is to test which interventions can move that residual, through model variants, training-protocol changes, and inference-time strategies.

The contributions are:

- an architecture-faithful, validation-locked replication that reproduces the paper’s recall and Hough pixel completeness (Chapter 5);
- a residual-error characterisation that localises the strict precision gap at the one-pixel boundary scale;
- controlled tests of a classifier-gated detector, an attention-gated U-Net, hard-negative mining, soft labels, ensembling, and probability-stratified Hough, each interpreted against that diagnosis;
- a reproducibility layer of preregistered selection rules, multi-seed reporting, and packaged, tested, documented code (Section 4.10).

Because the available data are 8-bit PNG renders with no surviving FITS, WCS, exposure, or photometric-calibration headers, this dissertation does not analyse sky-coordinate projection, horizon or zenith angle, or exposure-time-normalised trends (Section 2.1).

1.4 Code and data availability

The source code, configuration files, unit tests, and the locked model checkpoint are available in the GitHub repository, with rendered API documentation on Read the Docs. The MeerLICHT image and mask subset is collaboration data, provided through the MeerLICHT collaboration under the MeerLICHT data policy, and is not redistributed; the splits are reproducible from `src/data/splits.py` (seed 2804). The DECam frames used in Section 5.5 are public NSF NOIRLab Astro Data Archive products, identified by exposure and detector number in `results/classical/decam_cold_manifest.json`. All training and evaluation runs used the Cambridge Service for Data Driven Discovery (CSD3), operated by the University of Cambridge Research Computing Service.

Chapter 2

Dataset and Annotation Characteristics

2.1 The MeerLICHT subset

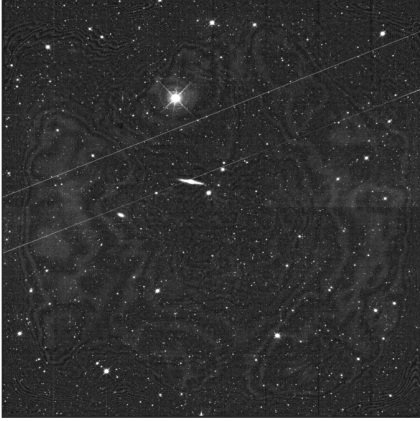
MeerLICHT is a dedicated 65 cm wide-field optical telescope in Sutherland, South Africa, designed to follow the radio sky in the optical band [4]. Its $10,560 \times 10,560$ -pixel CCD samples a 2.7 deg^2 field of view at $0.56'' \text{ pixel}^{-1}$, so a 528×528 patch subtends about $4.9'$ on a side. The labelled subset available for this work comprises 178 full-frame image-mask pairs: 8-bit PNG renders of FITS exposures, each paired with a hand-annotated binary trail mask (Figure 2.1). The PNG renders omit the original FITS headers, including WCS, exposure, and photometric-calibration metadata. This caps the per-pixel dynamic range, ruling out sky-coordinate, airmass, and exposure-time analyses. The data are therefore treated as a labelled display-space segmentation corpus, not as calibrated astronomical images.

2.2 Exploratory analysis

Exploratory analysis characterises the trail population and the number of labelled trail pixels per image, which the splits must balance. Annotated trails are long, near-linear features: connected components have small minor-axis widths and high aspect ratios, and many trails cross several adjacent patches. The per-image trail-pixel count is heavily skewed, which motivates the stratified split of Section 2.4 rather than a random split over images. Filename-derived observation dates span 2017–2023, dominated by 2022 exposures; because exposures and observing conditions are not normalised, these dates are purely descriptive.

Random Image and Mask Pairs

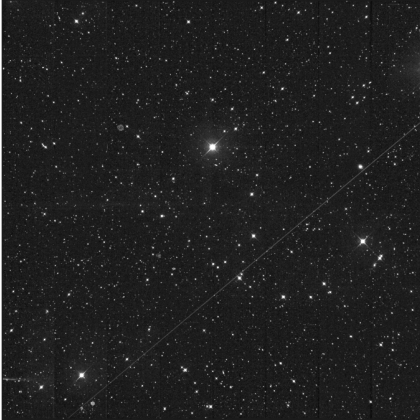
Image: ML1_20220728_180558_red.fits_full



Mask: ML1_20220728_180558_red_mask



Image: ML1_20191118_214959_red.fits_full



Mask: ML1_20191118_214959_red_mask

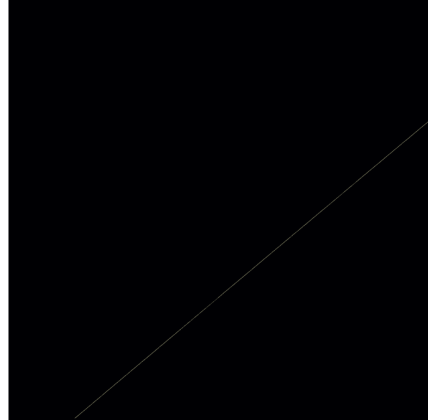


Figure 2.1: Example image–mask pairs from the labelled subset: 8-bit PNG renders with hand-annotated binary trail masks. Trails are long, near-linear features against a crowded stellar background.

2.3 Patch extraction and class imbalance

Full frames are too large for direct training. Patches are extracted at 528×528 pixels with stride 528, yielding exactly 400 non-overlapping patches per image (20×20 grid). The natural patch distribution is highly imbalanced, with approximately 93–94 % of full-grid patches containing no trail pixels. To form the sampled patch manifests (patch lists used by the training and evaluation code), all positive patches are retained and negatives are down-sampled to a 1:3 pos:neg ratio i.e., three negative patches drawn for each positive patch. Figure 2.2 gives the realised 528-pixel manifest sizes used by the final run.

The sampled test set is the main internal test population because it has the same 1:3 pos:neg prior as the validation set used for threshold selection. A separate parity test set keeps every test-frame patch, including all negatives, to restore the natural empty-patch prevalence for the paper-facing precision check (Figure 2.2); it is never used for model or threshold selection. This separation is crucial because adding many empty patches primarily changes precision: recall is computed over retained positive patches, whereas the additional negatives can introduce false positives.

2.4 Dataset splits

Figure 2.2 summarises the path from frames to evaluation manifests. Splits are performed at image level, never after patching, so every patch from a given frame remains in the same split. The 178 images are stratified by total trail-pixel count into four quartile groups, and the nominal 0.70/0.15/0.15 ratios are applied within each group with flooring; the remainder goes to test. The realised split is therefore **122 train** / **24 validation** / **32 test** (0.685/0.135/0.180), quoted throughout. This is leakage-safe and conservative, reserving a relatively large held-out fraction for evaluation, with splits seeded (2804).

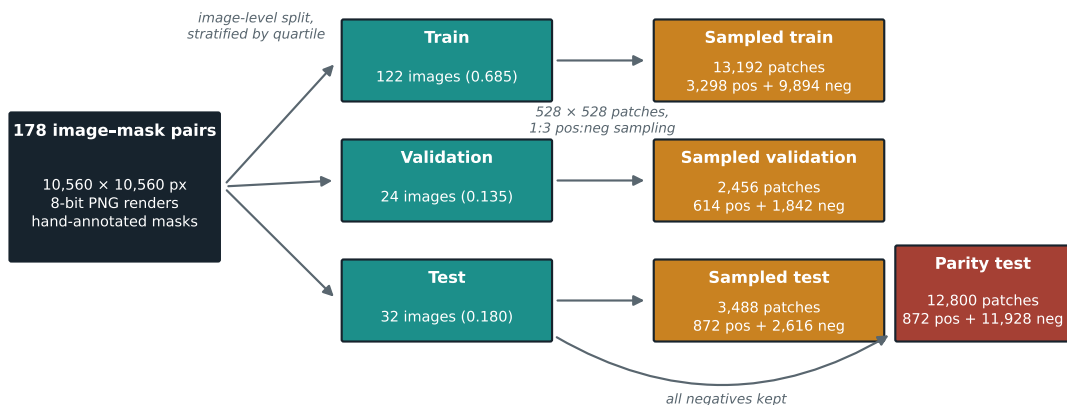


Figure 2.2: Data flow from the 178 labelled frames to the evaluation manifests. Sampled manifests drive training, threshold selection, and internal evaluation; the parity manifest restores all negatives for the paper-facing precision check and is never used for selection.

2.5 Mask quality

Beyond visual inspection, connected-component measurements provide a simple mask-quality check. Components have high aspect ratios and a median stroke width near 6 px, estimated from distances to the nearest mask edge, confirming genuinely linear structures; the densest patches adequately cover the visible trails (Figure 2.3).

The masks are sufficient for supervised training, with one characterised caveat: full-frame overlays show occasional mask under-coverage at faint endpoints and low-contrast stretches. If the model detects a visible trail segment omitted from the mask, those pixels are still counted as false positives, so strict label-referenced precision is a conservative estimate where masks omit visible trail pixels. Section 5.6.2 quantifies this boundary disagreement; interpretations are phrased as ‘consistent with’ boundary disagreement, not as proof of label error.

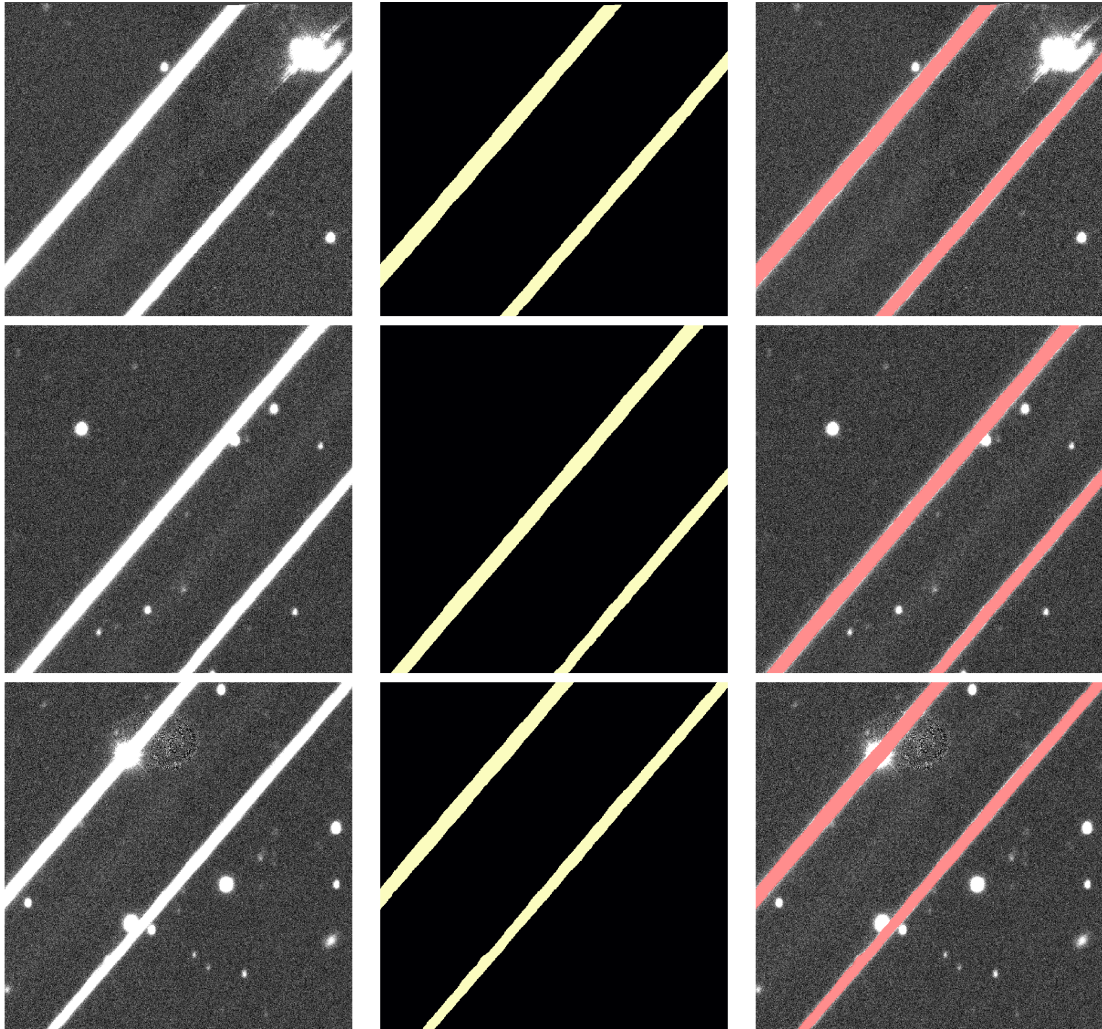


Figure 2.3: Mask-quality inspection grid for three of the densest patches: image, annotated mask, and overlay (red). The masks track the visible trails; the residual concern is mask under-coverage at faint endpoints, quantified in Section 5.6.2.

Chapter 3

Methodological Framing

3.1 Problem formulation

Trail detection is posed as patch-wise binary pixel segmentation. Given a 528×528 single-channel display-space patch, the model estimates a per-pixel probability that each pixel belongs to a satellite trail rather than background. Thus, the required output is a mask for downstream cleaning, not a catalogue of satellite identities or trail instances.

This is semantic segmentation: pixels are labelled by class, not assigned to individual trail instances. Crossing or near-parallel trails within a patch are simply “trail”, so pixel-level metrics cannot distinguish merged from separated trails. The formulation is label-referenced: success is measured against the available binary masks, so boundary placement and mask under-coverage impact precision.

3.2 Learned segmentation with geometric post-processing

The pipeline pairs a learned segmentation stage with a classical geometric stage. A U-Net produces a soft-detection map that preserves weak, spatially extended evidence of a faint trail. A classical Hough detector applied directly to the raw image responds mainly to stars, galaxies, and diffraction structure. In a validation-tuned raw-image Hough implementation, precision was only about 10^{-3} , confirming that raw-image Hough voting is an insufficient masking baseline. Thus, the probabilistic Hough transform is used after the U-Net, where the input is already a trail-probability canvas rather than a crowded sky image. There, it can exploit the prior that a trail is globally linear to bridge gaps and recover continuous trail structure. The remaining methodological choices follow this framing: the U-Net supplies pixel-level segmentation, the BCE–Dice loss handles severe foreground imbalance, and evaluation protocols separate selection, prevalence, and label-boundary effects.

3.3 U-Net architecture choice

The U-Net encoder–decoder with skip connections [23] preserves full spatial resolution. This is essential for thin, elongated, sparse targets, while its encoder path still supplies enough context to distinguish trails from stellar structure and background gradients. The same architecture underpins close astronomical artefact detectors such as deepCR [37] and MaxiMask [22], supporting its suitability here.

More elaborate variants are treated as follow-up tests rather than replacements for the replication baseline; Section 5.7.2 tests whether adding attention gates changes the residual error pattern. Transformer-style segmenters, such as ViT, are not explored as replication baselines because they would change the detector family and introduce data and tuning demands that are poorly matched to this relatively small dataset.

3.4 Combined BCE and Dice loss

The foreground occupies a tiny fraction of pixels, so a pure cross-entropy objective is dominated by easy background. The Dice term measures region overlap and is less dominated by the background majority [18]. Combining it with binary cross-entropy keeps per-pixel calibration while following the Combo-loss pattern for severely imbalanced segmentation [29]. Binary cross-entropy also allows a positive-class weight, `pos_weight`, to penalise missed trail pixels further. However, this was not applied in the final pipeline as the 1:3 patch sampling and Dice term already address the imbalance. Stacking another class weight would bias the model towards over-prediction.

3.5 Evaluation protocol and assumptions

Four principles govern the evaluation. Model and threshold selection are pre-registered and validation-only, so no test metric is inspected before the configuration is locked. Sampled and all-negative test metrics are kept distinct: the sampled set matches the validation prior, while the all-negative set restores the natural empty-patch prevalence used for the paper-facing comparison. Uncertainty treats the source image, not the patch, as the independent unit, because patches from one frame share background, atmospheric image quality, and annotation structure. Finally, because masks omit some visible trail pixels (Section 2.5), strict pixel-overlap scores are complemented by topology-aware (clDice), false-positive distance, and boundary-tolerant metrics (Section 4.8).

Chapter 4

Model Implementation and Training Protocol

4.1 Data pipeline

The on-disk patch builder implements the extraction scheme from Section 2.3: it writes 528×528 stride-528 patches at the 1:3 sampling ratio, stored in their original extracted orientation. Each manifest row records the source image, split, patch coordinates, positive-pixel fraction, and full-image display-space mean and standard deviation. The last two fields implement the `full_image` normalisation mode: each patch is converted to $[0, 1]$ and normalised with the statistics of its parent image, matching the paper’s per-image standardisation rather than normalising each patch independently.

4.1.1 Data augmentation

Training patches are augmented at load time with random horizontal flips, vertical flips, and $k \times 90^\circ$ rotations applied identically to each image and mask. Stored patches therefore keep their original extracted orientation while appearing in different orientations across epochs. Validation and test patches are not augmented. Training images also receive signal-dependent stochastic noise; masks and evaluation splits do not. This is not modelled as Poisson–Gaussian CCD noise since the inputs are display renders rather than calibrated electron counts. For a patch-pixel value $p \in [0, 1]$ the augmentation samples

$$p_{\text{noisy}} = \text{clip}\left(p + \mathcal{N}\left(0, m\sqrt{\alpha p + \beta^2}\right), 0, 1\right),$$

where m is the noise multiplier and α, β are calibrated once from empty-mask training patches by `scripts/data/compute.background.noise.stats.py`, so the injected noise matches the measured background variation. The locked calibration uses 9,894 empty train patches, giving $\alpha = 0.0209$ and $\beta = 0.0886$. The active configuration fixes $m =$

1.0; a preliminary three-seed screen compared a no-noise control with multipliers $m \in \{0.5, 1.0, 2.0\}$. The $m = 1.0$ setting was kept for robustness, although a validation- F_1 gain was not decisive.

4.2 Network architecture

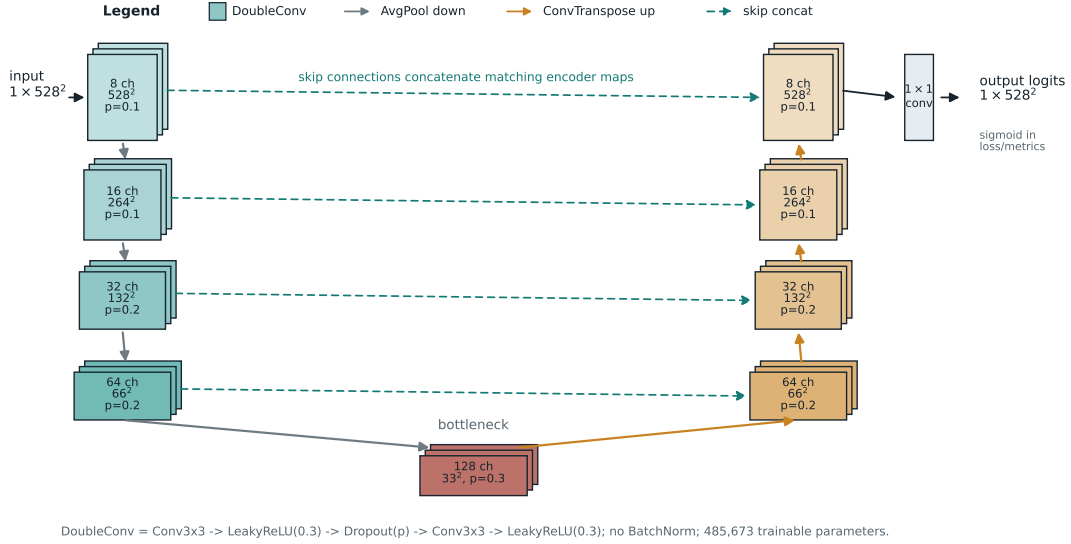


Figure 4.1: Architecture-faithful U-Net. Encoder widths $8 \rightarrow 16 \rightarrow 32 \rightarrow 64$ with a 128-channel bottleneck, average-pooling downsampling, transposed-convolution upsampling, and skip concatenation at each level. Each block is two 3×3 convolutions with LeakyReLU(0.3) and in-block dropout.

The replication network (Figure 4.1) was reconstructed from the released Stoppa et al. Keras `model-best.h5` configuration: an encoder-decoder with skip connections and filters progressing $8 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 128$, average-pooling downsampling, transposed-convolution upsampling, and LeakyReLU activations with slope 0.3. Each double-convolution block follows `Conv2d` → `LeakyReLU` → `Dropout` → `Conv2d` → `LeakyReLU`, with element-wise dropout in all nine blocks. The dropout rates from stem to final decoder block are

$$0.1, 0.1, 0.2, 0.2, 0.3, 0.2, 0.2, 0.1, 0.1.$$

The network uses no batch normalisation. The 3×3 convolutions use He-normal initialisation [11]; the transposed convolutions and the final 1×1 head use Glorot-uniform initialisation; all biases are initialised to zero. Inputs are 528×528 single-channel patches and the output is a single-channel logit map with identical spatial dimensions. Sigmoid conversion is applied outside the model in the loss, thresholding, Hough, and visualisation paths. Altogether, the network has 485,673 trainable parameters (`base_channels = 8`).

4.3 Loss function

Training uses a combined binary cross-entropy (BCE) and squared-Dice loss:

$$\mathcal{L} = \lambda_{\text{BCE}} \mathcal{L}_{\text{BCE}} + \lambda_{\text{Dice}} \mathcal{L}_{\text{Dice}}, \quad \lambda_{\text{Dice}} = 1 - \lambda_{\text{BCE}}, \quad (4.1)$$

where

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2 \sum_i p_i g_i + \epsilon}{\sum_i p_i^2 + \sum_i g_i^2 + \epsilon}, \quad \epsilon = 10^{-4},$$

with p_i the sigmoid probabilities and g_i the binary targets. The Dice term follows Milletari et al. [18]; combining it with BCE follows the Combo-loss family of Taghanaki et al. [29]. The BCE term uses `BCEWithLogitsLoss` without a class-balancing `pos_weight`, matching the paper-faithful path and avoiding a second imbalance correction on top of the sampled patch prior. Losses are evaluated in FP32 even though the forward pass uses BF16 autocast [14, 17], avoiding low-precision accumulation in the BCE and Dice reductions.

4.4 Hyperparameter search

Hyperparameter search uses Optuna [1] with a tree-structured Parzen estimator and median pruner ($n_{\text{startup}} = 15$, 10 warm-up epochs). The 45 trials are balanced across batch sizes $\{8, 16, 32\}$, with 15 trials assigned to each batch size. Within each batch-size stratum, the continuous hyperparameters are learning rate and BCE weight λ_{BCE} ; dropout, normalisation, and noise multiplier are fixed. Appendix A gives the full search-space and fixed-configuration table. Each trial runs for 20 epochs and is scored by threshold-swept validation F_1 , matching the final selection criterion. Pruning uses in-epoch validation Dice at the fixed training threshold as an early-pruning signal; only completed trials are rescored by swept validation F_1 . The top five trials by validation F_1 are retrained for 75 epochs at five seeds (2804, 1234, 42, 7, 13), with the winner chosen by mean threshold-swept validation F_1 across seeds *before any test metric is inspected*. The preregistered tie-breaker, applied when means fall within 0.001 validation F_1 , prefers simpler/lower-risk configurations: lower batch size, then lower learning rate, then lower trial index. The selected base U-Net is trial 44 (`batch.size=32`, learning rate 2.89×10^{-4} , $\lambda_{\text{BCE}} = 0.655$), with mean validation $F_1 = 0.854$ across the five retrained seeds; trial 32 reached a marginally higher raw mean (0.855), but trial 44 won the within-0.001 tie band on the lower-learning-rate rule.

4.5 Threshold selection

A precision–recall curve is swept on the validation set to select the F_1 -optimal threshold, which is applied unchanged to the test evaluation. Each reported seed uses its own validation-selected threshold, with 0.45 for the primary seed 2804 (the paper fixes 0.58). Within each run, the saved best epoch is selected by validation Dice at the fixed training threshold. That checkpoint is then swept over thresholds on the validation set, and the resulting best validation F_1 is used for trial ranking and for choosing the probability threshold used at test time.

4.6 Hough post-processing

A probabilistic Hough transform [16] (`cv2.HoughLinesP`) is applied to a lower-threshold probability canvas to recover faint, linear detections below the segmentation threshold. For each test image the per-patch sigmoid probability maps are max-pooled into a canvas before Hough voting. The sampled evaluation uses the 1:3 patch support, while the parity Hough comparison uses full 528 px tiling over every test image. An independent full-coverage tiled pass reproduced that parity canvas pixel-for-pixel, ruling out a patch-sampling or seam gap. The canvas is thresholded at $t_{\text{Hough}} = 0.1$, after which `HoughLinesP` is run with a vote threshold of 50, a minimum line length of 100 px, and a maximum gap of 250 px, matching the released ASTA reference implementation from Stoppa et al. [27]. Detected segments are drawn back onto the canvas at 3 px thickness, making line-rasterisation convention a primary comparability caveat for strict precision.

4.7 Patch CNN classifier for the two-stage variant

A two-stage variant tests whether an inexpensive patch-level gate can reduce empty-patch false positives and U-Net inference cost without losing trail-bearing patches. Section 5.7.1 explores placing a lightweight convolutional neural network (CNN) patch classifier in front of the U-Net. The classifier is intentionally simple: a four-block strided encoder followed by global average pooling and a single fully connected logit head. It is trained on the sampled training manifest with labels derived from the patch positive-pixel fraction and a `pos.weight` equal to the train-split negative-to-positive ratio. Validation reports both the maximum- F_1 and a high-recall (recall ≥ 0.99) operating point, because a patch rejected by the gate is unrecoverable downstream. The classifier and the U-Net are trained independently and composed only at inference, keeping the gate a single-variable change to the locked pipeline. The classifier never sees test data during U-Net selection.

4.8 Evaluation metrics

Segmentation quality is reported with pixel precision, recall, Dice (equal to F_1 for binary masks), and IoU. Because trails are thin connected structures, pixel overlap alone can miss whether a prediction preserves centreline continuity. Topology is therefore assessed with the centreline Dice (clDice) [25]. Boundary disagreement is isolated with a boundary-tolerant precision/recall/ F_1 score at $k \in \{0, 1, 2, 3\}$ px, where $k = 0$ is the strict metric and $k > 0$ allows a predicted pixel and a ground-truth pixel to match within a small spatial tolerance. The spatial structure of false positives is summarised by their distance to the nearest ground-truth trail pixel, with whole-patch false positives reported separately because they have no defined trail distance. Uncertainty is quantified primarily by a cluster bootstrap over source images, resampling images with replacement and aggregating their patches before recomputing each metric. Patch-level bootstrap is reported only as a secondary diagnostic.

4.9 Determinism and reproducibility

All experiments seed Python, NumPy, PyTorch, and DataLoader workers. CUDA settings use `cudnn.deterministic=true` and `cudnn.benchmark=false` ; the cuBLAS workspace is fixed, and deterministic PyTorch algorithms are requested with warnings rather than hard failures. Residual GPU non-determinism is absorbed by five-seed retraining of the top trials and seed-averaged reporting. The probabilistic Hough stage (`cv2.HoughLinesP`) is the one post-hoc step whose OpenCV RNG is not separately seeded, so its reruns are near- but not bit-identical.

4.10 Reproducibility and software structure

The code is packaged as an installable Python project with module-level command-line interfaces for each pipeline stage. Experiments use YAML configuration plus runtime overrides, so runs are reproducible from configuration and seed.

Correctness is checked by a `pytest` suite covering image and mask pairing, patch geometry, the recovered architecture, loss and segmentation metrics, the shared Hough runner, and training entry points. Ruff handles static checks. Separate CUDA and CPU requirements are maintained, and a portable CPU Docker image runs the project. NumPy-style docstrings are rendered with Sphinx and served on ReadTheDocs.

Chapter 5

Replication and Error Diagnosis

5.1 Replication of the target detector

The validation-selected winner (trial 44; Section 5.2) is evaluated on the held-out test images at its selected threshold of 0.45. On the sampled test set (1:3 pos:neg) the single-seed model achieved precision 0.852, recall 0.918, and Dice 0.884. The five-seed models achieved a mean of precision 0.847 and recall 0.923 at their respective validation-selected thresholds (Table 5.1, Figure 5.1). On the parity test set, precision falls to 0.780 at unchanged recall (0.918), as expected once the negative base rate is restored. Recall is therefore reproduced at close to the paper’s level, while strict precision sits below the paper’s 0.94. The rest of the chapter analyses this gap rather than treating it as an optimisation target.

Table 5.1: Replication comparison against Stoppa et al. (2024). Paper metrics are at threshold 0.58 on a 20,000-patch test set; this work uses the realised 122/24/32 image split and the validation-selected threshold 0.45 (per-seed 0.41–0.50). Cluster-bootstrap intervals are in Section 5.4. Dashes indicate exact values were not provided in the paper.

Run	Threshold	Precision	Recall	Dice	IoU
Stoppa et al. (2024)	0.58	0.94	0.94	—	—
This work, sampled (single seed)	0.45	0.852	0.918	0.884	0.791
This work, sampled (5-seed mean)	0.41–0.50	0.847	0.923	0.884	0.792
This work, parity (single seed)	0.45	0.780	0.918	0.843	0.729

5.2 Validation-only model selection

Trial 44 was selected by mean validation F_1 across the five-seed retraining of the top five trials, before any test metric was inspected (Section 4.4; Figure 5.3). Its validation precision–recall sweep sets the operating threshold of 0.45, carried unchanged into the

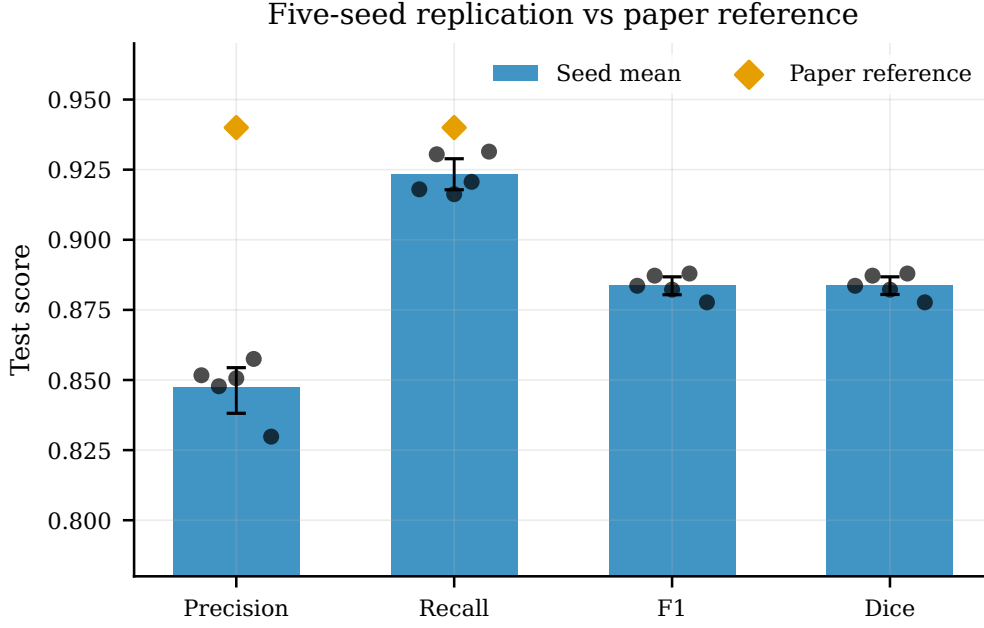


Figure 5.1: Five-seed replication summary for the locked validation-selected U-Net configuration. Paper-reference markers are for the explicitly reported precision and recall values.

primary seed-2804 test evaluation. Across the five reported seeds, the validation-selected thresholds range from 0.41 to 0.50, and each seed is evaluated at its own validation-selected threshold. Figure 5.2 shows the validation sweep for the primary seed; the F_1 plateau is flat between roughly 0.4 and 0.6, so small threshold shifts would not materially affect validation F_1 .

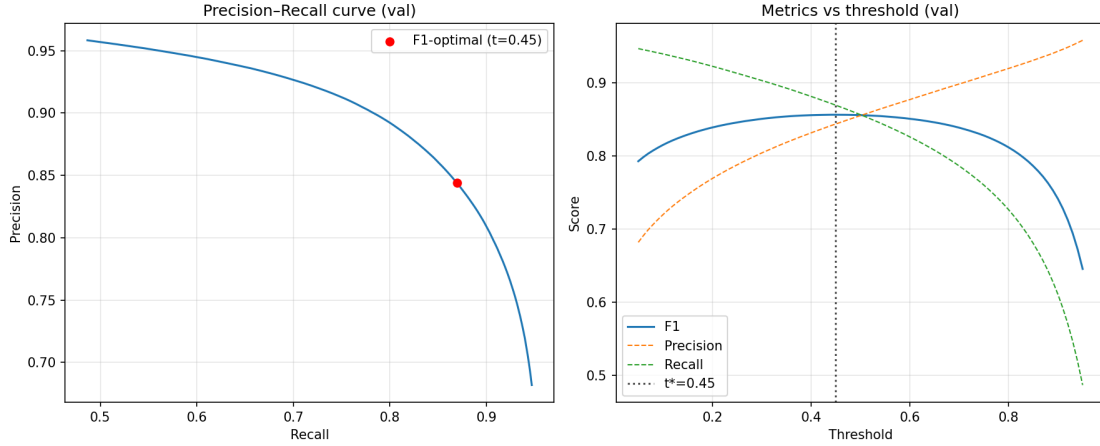


Figure 5.2: Validation-only sweep for the primary seed-2804 winner: PR curve with the F_1 -optimal point ($t^* = 0.45$, left) and metrics versus threshold (right). The vertical line marks the selected threshold; the flat F_1 plateau spans the paper’s fixed 0.58.

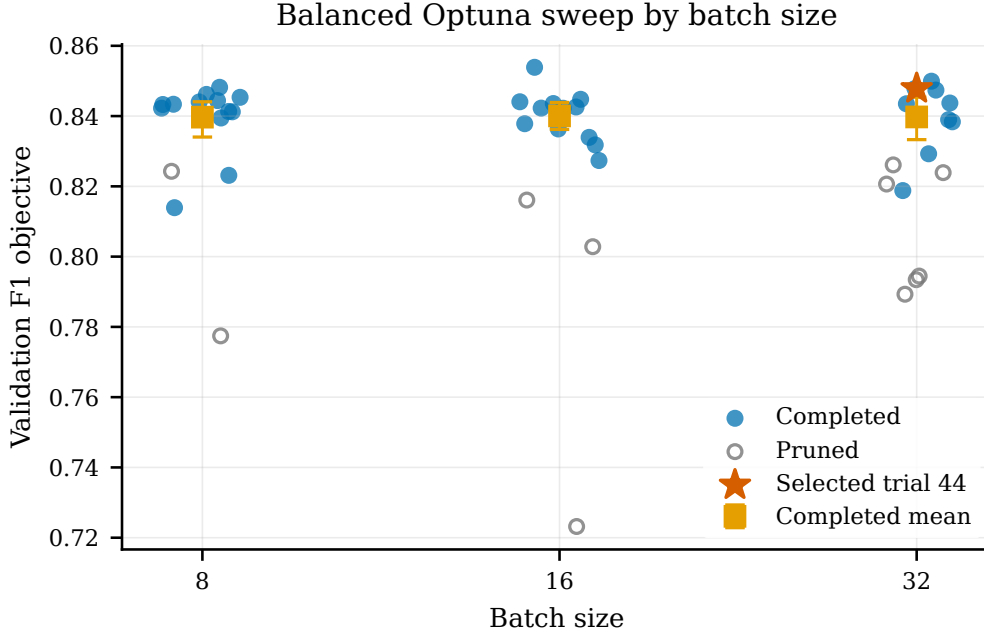


Figure 5.3: Balanced Optuna sweep by batch size. Blue markers are completed trials, hollow grey markers are pruned trials, orange squares show completed-trial means, and the star marks trial 44, selected after validation-only top-five retraining and tie-breaking.

5.3 Hough improves pixel completeness

Probabilistic Hough post-processing raises trail-pixel recall from 0.918 to 0.990 for the primary seed on the full-coverage parity canvas, equivalently reducing the pixel-level false-negative rate from 0.082 to 0.0105 (five-seed mean $0.077 \rightarrow 0.0098$). This pixel-completeness rate is comparable to the paper’s 6.98 % $\rightarrow$ 3.38 %. The full-coverage tiled pass described in Section 4.6 reproduced the parity canvas pixel-for-pixel, ruling out a patch-sampling or seam artefact. The image-level detection false-negative rate is degenerate here: every positive test image is already hit by at least one predicted pixel, so the rate is 0 both before and after Hough and is not comparable to the paper’s image-level rate. At patch level, the rate falls only from 0.122 to 0.103, since this stricter diagnostic counts trail-bearing patches with no recovered pixel. Thus, Hough is treated as a completeness mechanism rather than a precision fix.

Figure 5.4 shows Hough in action: the U-Net misses a trail stretch over a bright star, and Hough recovers ground-truth pixels that appear only after the geometric stage.

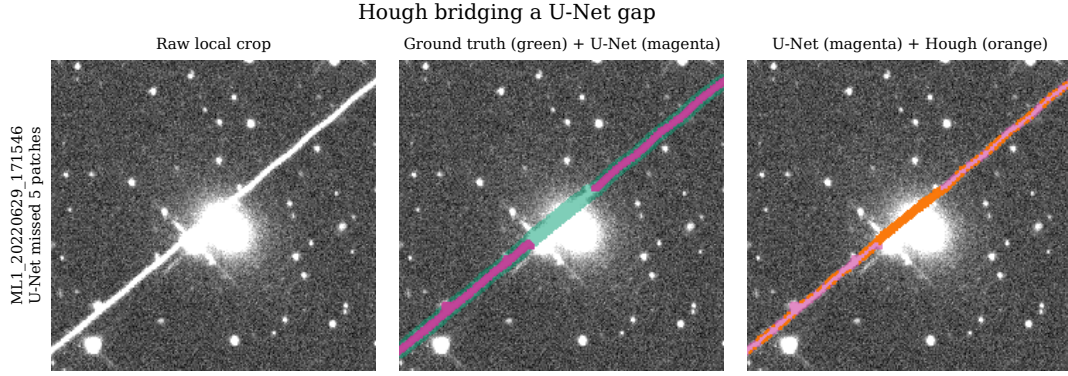


Figure 5.4: Hough bridging a U-Net gap. Panels: raw image; ground truth (green) with U-Net prediction (magenta); U-Net (magenta) with Hough bridge (orange). Ground-truth pixels over the bright star are recovered only by Hough.

5.4 Uncertainty quantification

Using a cluster bootstrap over the 32 held-out test images (1,000 resamples, single-seed winner), the 95 % intervals are precision 0.852 [0.806, 0.891], recall 0.918 [0.888, 0.946], Dice 0.884 [0.857, 0.904], and IoU 0.791 [0.749, 0.825]. These image-level intervals are several times wider than the patch-level bootstrap (Dice [0.876, 0.892]). This is reported only as a secondary diagnostic since patches from the same exposure share background/trail structure and should not be treated as independent observations. The image-cluster interval is *the* headline interval; the patch bootstrap is included only to show how much patch-level resampling would understate uncertainty. Across the five trial-44 seeds, the sampled-test means and sample standard deviations are precision 0.847 ± 0.010 , recall 0.923 ± 0.007 , and Dice 0.884 ± 0.004 . So, seed variation is smaller than image-cluster uncertainty, though still visible.

5.5 Qualitative prediction checks

A selection of test predictions is shown in Figure 5.5; Appendix B.2 shows empty-mask false positives and matched high-Dice true positives, followed by the same eight patches for the attention variant. They include diffraction-spike examples: low-Dice false positives on stellar structure and high-Dice cases where bright stars are correctly ignored.

The model was also applied to nine DECam frames with RECA-measured streaks (Figure 5.6). With no masks, no metric is reported; on manual visual review, 8/9 visible streaks are fully traced with a faint streak partially traced and counted as a miss. This is consistent with the low-contrast behaviour described in Section 5.6.4 and illustrates out-of-domain behaviour rather than a cross-dataset benchmark.

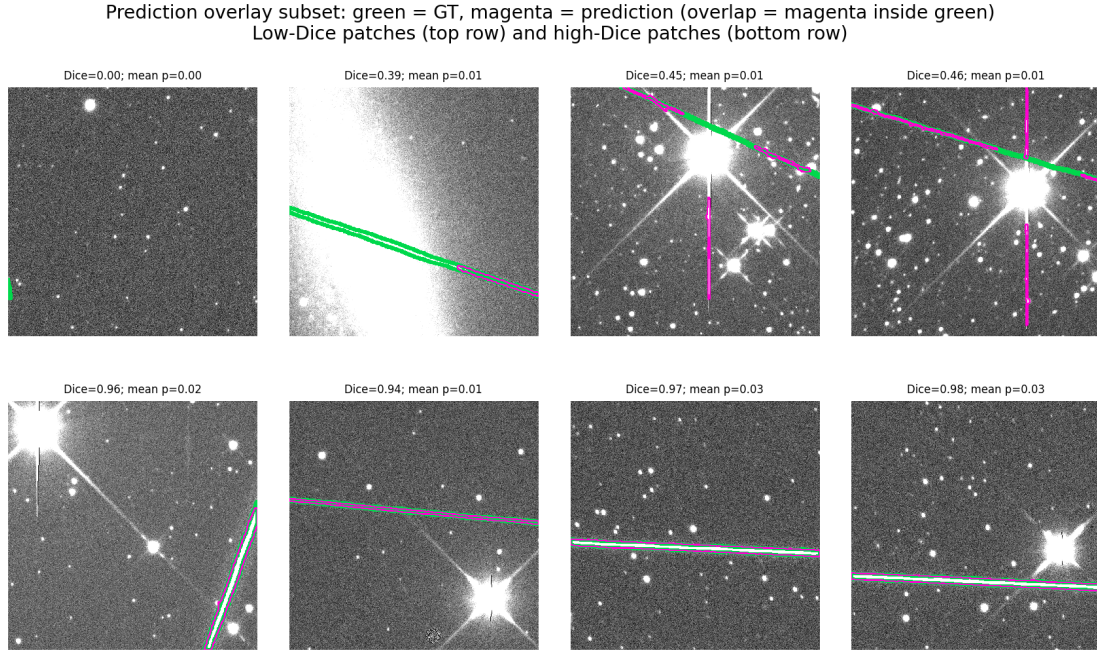


Figure 5.5: Diagnostic selection of test-patch predictions for the locked winner: image, ground-truth mask (green), and predicted mask (magenta), with overlap shown as magenta inside the green band.

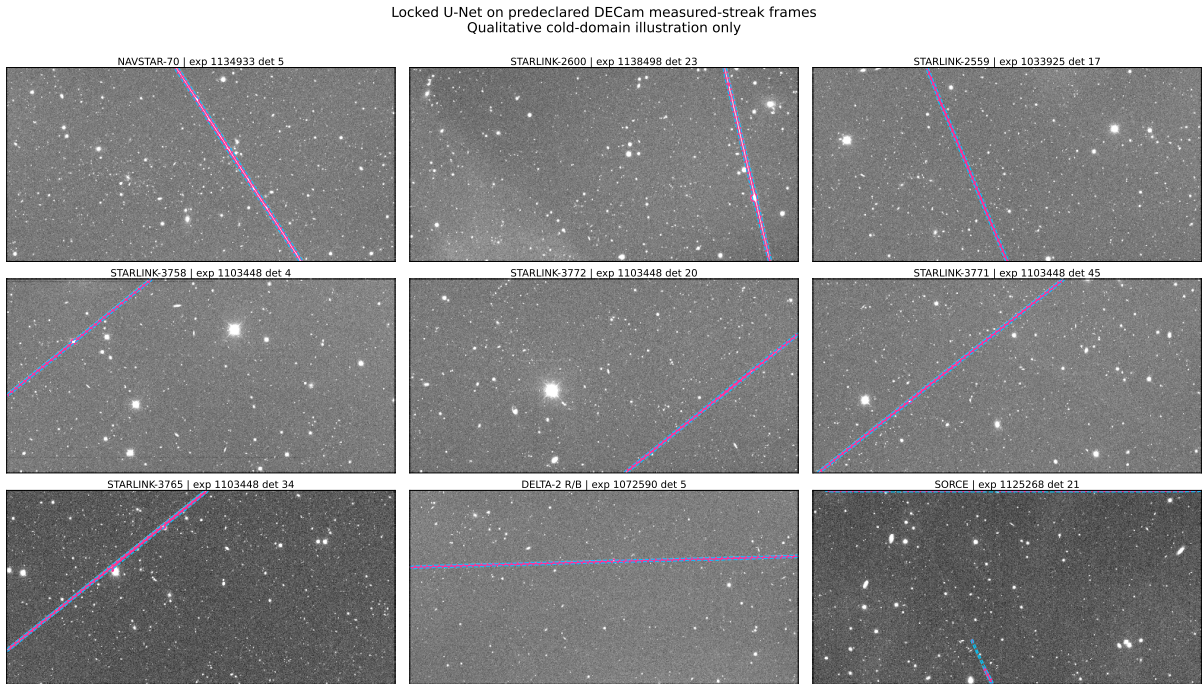


Figure 5.6: Locked MeerLICHT-trained U-Net applied cold to predeclared DECam measured-streak frames (magenta: U-Net contour at $p \geq 0.45$; cyan dashed: Hough). Qualitative illustration only: no ground-truth masks or benchmark metrics exist.

5.6 Diagnostics localise much of the precision gap near boundaries

The diagnostics below separate where the residual false positives occur from why they occur. Most false-positive mass is in trail-bearing patches, and the distance analysis shows that a majority of false-positive pixels lie within one pixel of labelled trails. This is consistent with thin or imperfect mask boundaries, while the smaller inter-patch component consists of predictions in labelled trail-free patches which cannot be explained by mask-boundary placement. Section 5.6.6 tests this reading against a blinded single-author re-annotation.

5.6.1 False-positive decomposition

Decomposing the false-positive pixels of the winning model shows that only 17.5 % are inter-patch (the network lighting up in trail-free patches), while 82.5 % are intra-patch (falling inside trail-bearing patches, though not necessarily adjacent to the trail) (Table 5.2). A patch-level gate can only remove the inter-patch share, making 17.5 % a hard upper bound for the gate. Hard-negative mining targets the same failure mode during retraining, so this value is a rough scale for the possible effect.

Table 5.2: Decomposition of false-positive pixels. Only the inter-patch component is directly removable by a patch-level gate. For hard-negative mining, this gives a scale of the empty-patch failure mode being targeted.

Component	Pixels	Share
Intra-patch FP	300,745	82.5%
Inter-patch FP	63,662	17.5%

5.6.2 Boundary-tolerant evaluation

Re-scoring under a symmetric boundary tolerance of $k \in \{0, 1, 2, 3\}$ px (RECT structuring element; a predicted pixel within k px of ground truth counts towards precision, and a ground-truth pixel within k px of a prediction counts towards recall) tests how much of the strict score is sensitive to boundary placement rather than larger misses (Figure 5.7). Allowing a single pixel of tolerance, the five-seed mean precision rises from 0.847 (strict) to 0.946, recall from 0.923 to 0.980, and the ± 1 px F_1 reaches 0.962: the precision gap to the paper largely closes under a one-pixel tolerance. A one-sided check separates this from a recall-for-precision trade: re-scoring the same predictions against ground truth dilated by one pixel raises precision from 0.852 to 0.937 but drops recall to 0.793, whereas the symmetric tolerance raises both. This is consistent with much of the residual error

occurring at boundary scale rather than as detections far from any labelled trail. The blinded re-annotation of Section 5.6.6 supports this reading. The tolerant score is therefore used only as a diagnostic: it relaxes the pixel-matching rule and is not directly comparable to the paper’s 0.94/0.94.

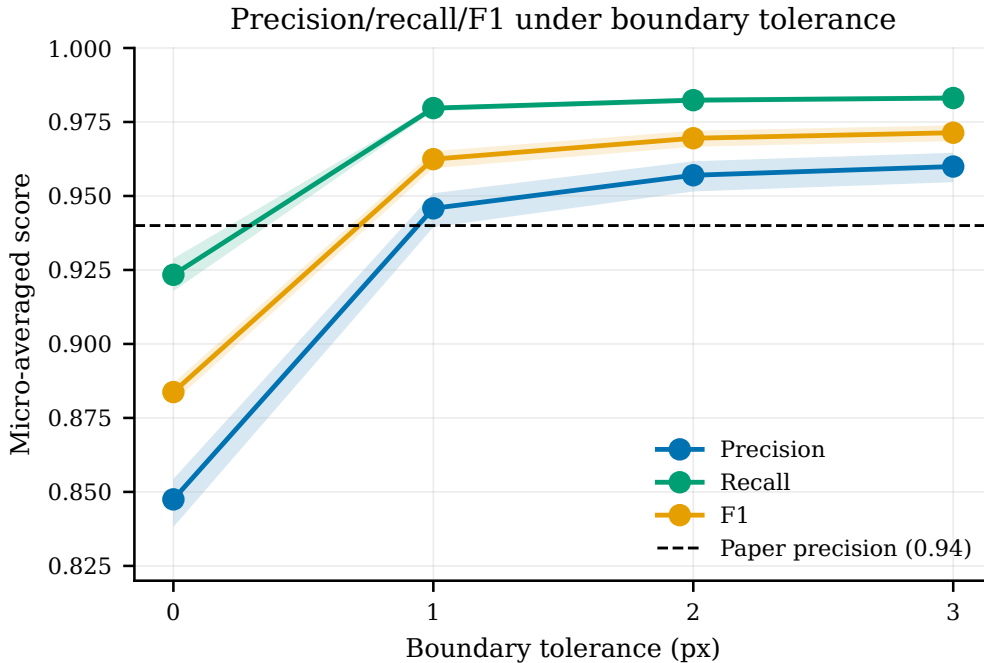


Figure 5.7: Boundary-tolerant metrics versus tolerance k (micro-averaged); the horizontal line marks the paper’s strict precision of 0.94. One pixel of tolerance largely closes the strict gap, but the relaxed score is not directly comparable to the paper’s strict metric.

5.6.3 False-positive distance to mask

The distance from each false-positive pixel to the nearest ground-truth trail pixel concentrates at small values (Figure 5.8). Of all false positives, 57 % lie within 1 px; of the distance-defined false positives (those in trail-bearing patches, excluding the $\sim 17\%$ whole-patch false positives that have no defined distance), 69 % lie within 1 px, with a median of 1.6 px. Both denominators are reported because the 69 % value excludes whole-patch false positives, so it would overstate the share of all false positives that are boundary-near. Together, these denominators indicate that many false positives are boundary-near, while a non-negligible minority remains away from any labelled trail.

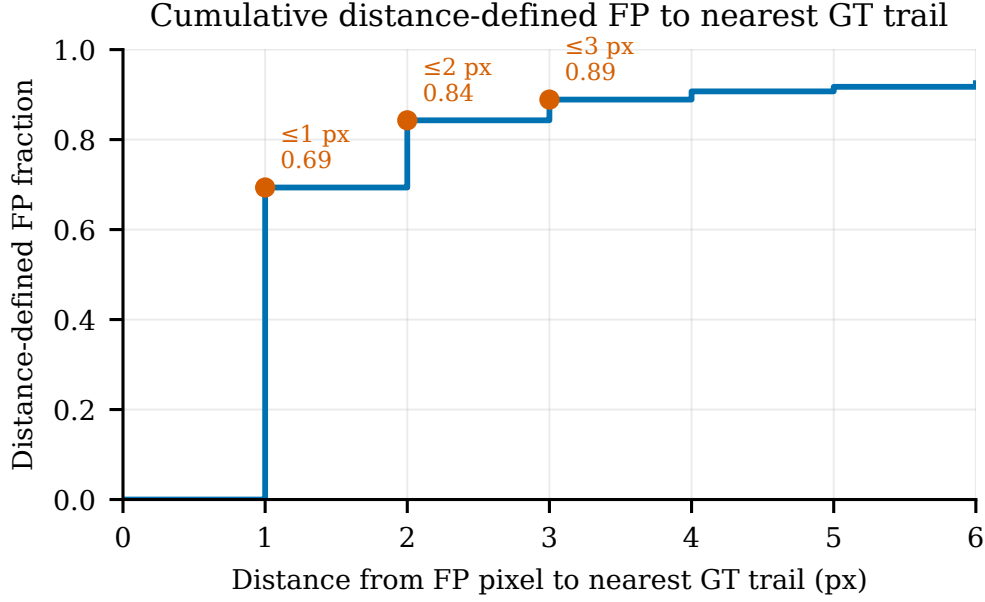


Figure 5.8: Cumulative fraction of distance-defined false-positive pixels within distance d of the nearest ground-truth trail. The plotted curve excludes 63,662 whole-patch FP pixels; 69.3 % of distance-defined and 57.2 % of all FP pixels lie within 1 px.

5.6.4 Relative display-contrast diagnostics

The locked winner was rescored post hoc on the 31 positive test images using raw 8-bit PNG intensities as a *relative display-contrast proxy*, not calibrated flux or signal-to-noise (Appendix Figures B.1–B.3). Median cross-sections through labelled trails illustrate the proxy (Figure B.2), but calibrated flux measurements would require original count data that the 8-bit renders do not carry (Section 2.1). Across 55 cleaned trail components divided into three approximately equal bins by relative display contrast, the lowest-contrast bin has the weakest pre-Hough recall (0.853) and the largest additional ground-truth recovery from Hough (0.092). After Hough, recall is at least 0.945 in all three bins. Near-trail false positives have median display excess between the background and labelled-trail medians (Figure B.3); this is compatible with some boundary under-coverage, but remains a descriptive diagnostic rather than evidence of label error.

5.6.5 Topology-aware comparison (clDice)

As a topology check, clDice asks whether the predictions preserve the trail centreline structure, rather than only overlapping the labelled pixels. The centreline Dice tracks pixel Dice and preserves the architecture ordering: 0.847 for the base model versus 0.838 for the attention variant (-0.009), against pixel Dice 0.884 and 0.879 (-0.005). Thus, the attention model does not appear to recover more continuous trail structure that is hidden by the pixel-overlap metrics.

5.6.6 Blinded re-annotation audit

The diagnostics above localise the residual error but leave two explanations: visible trails may be wider than their masks, or the model may systematically over-paint. To separate these, a single-author, evaluation-only re-annotation was performed on 64 sealed 528×528 crops. Stratified by display contrast and comprising interior and endpoint segments for boundary scoring plus false-positive-prone and quiet decoy patches as existence controls.

The audit does not support a systematic width bias. The original MeerLICHT masks, the blinded re-annotation, and the model prediction all have median stroke width 6 px. Per-crop width differences are centred near zero: re-annotation minus original is +0.10 px (95% CI $[-0.30, +0.53]$), and model minus re-annotation is +0.22 px ($[-0.09, +0.54]$). The strict false-positive gap therefore cannot be explained simply by labels being uniformly too thin or by the model being uniformly too wide.

Pixel agreement gives the same strict-tolerance picture. Over the 44 interior and endpoint crops, the model matches the re-annotation ($F_1 = 0.893$) about as closely as the original labels do (0.890), while the model and original labels score 0.894 against each other (Table 5.3, pixel-weighted micro-averages). A paired per-crop bootstrap ($B = 10,000$, equal weight per crop) puts the strict model-minus-original difference against the re-annotation at -0.0001 (95% CI $[-0.024, +0.022]$). At ± 1 px the model is marginally lower ($\Delta = -0.015$, CI $[-0.038, +0.003]$, just spanning zero). Because the model was trained on the original labels, this is not an independent confirmation of the model. For this sample, model-reannotation agreement is not statistically distinguishable from original-label-reannotation agreement under strict scoring. The pattern is stable across audit groups, with median widths of 6 px and strict F_1 near 0.86 on interior segments and 0.89 on endpoints.

Table 5.3: Blinded re-annotation audit: micro-averaged strict and ± 1 px F_1 over the 44 interior and endpoint crops. All three sources share a median stroke width of 6 px.

Comparison	strict F_1	± 1 px F_1
Original labels vs re-annotation	0.890	0.978
Model vs re-annotation	0.893	0.970
Model vs original labels	0.894	0.972

The control crops show a small tendency for the model to mark signal where the re-annotation does not. The model detects signal on all 11 false-positive-prone controls, while the re-annotation marks a trail on only 5; on the six over-fired crops the median extra area is 134 px, about 0.05 % of a crop. By contrast, the quiet decoy crops produce no model detections. Figure B.6 shows overlays and Figure B.7 documents the annotation process. Since this audit had only one annotator, it measures disagreement between the original labels, the re-annotation, and the model, not inter-annotator agreement. Its value

is in testing whether a large width mismatch is visible, not what the unique true mask should be. A multi-annotator audit remains the most valuable check (Section 7.2).

5.7 Model variants as diagnostic checks

Model variants test the error diagnosis. If the precision gap were dominated by empty-patch false positives, a patch classifier or hard-negative training should improve precision; if by missing faint structure, attention gates might improve overlap or topology. The observed results are narrower: the gate shifts precision consistent with the 17.5% inter-patch bound, and attention does not improve the base architecture.

5.7.1 Classifier-gated detector

The classifier-gated detector uses the CNN described in Section 4.7. It is reported as an operating-characteristic Pareto front over the classifier and U-Net thresholds (Figure 5.9), not as a single selected replacement for the locked U-Net. The gate suppresses only empty-patch false positives, so its gain is bounded by the inter-patch share. At the illustrative high-recall operating point used for Figure 5.9 ($t_{\text{clf}} = 0.18$, $t_{\text{U-Net}} = 0.45$), the gate routes 2,625 patches to the U-Net (75.3% of the sampled test set), improving precision from 0.852 to 0.864 while reducing recall from 0.918 to 0.914 and giving Dice 0.888. Recall counts classifier-rejected positive patches as false negatives, exposing the classifier false-negative rate (0.096 at this operating point) as a hard recall ceiling. The gate gives a bounded precision gain (the +1.2 pp shift is about one five-seed precision SD) and a real compute saving at this operating point, but not an order-of-magnitude saving: it skips 24.7% of sampled-test U-Net forwards. Greater savings would be expected on a full-grid or production stream, where empty patches dominate.

5.7.2 Attention-gated U-Net

The attention-gated U-Net [21] is a near-capacity-matched variant, carrying roughly 1% more parameters than the base U-Net. Under the matched search and validation-only selection protocol of Section 4.4, it does not improve on the base network: the selected attention model reaches test precision 0.838, recall 0.924, and Dice 0.879 at threshold 0.48, against the base Dice of 0.884 (Table 5.4). The five-seed validation-restudy means are 0.854 for the base model (90% interval [0.851, 0.856]) and 0.851 for the attention model ([0.850, 0.852]), so the comparison is best read as matched-protocol null evidence rather than as a decisive architectural ranking. Together with the clDice result (Section 5.6.5), this gives no evidence that attention recovers more continuous or faint trail structure on this dataset. Supplementary Figure B.9 shows the attention-model gallery.

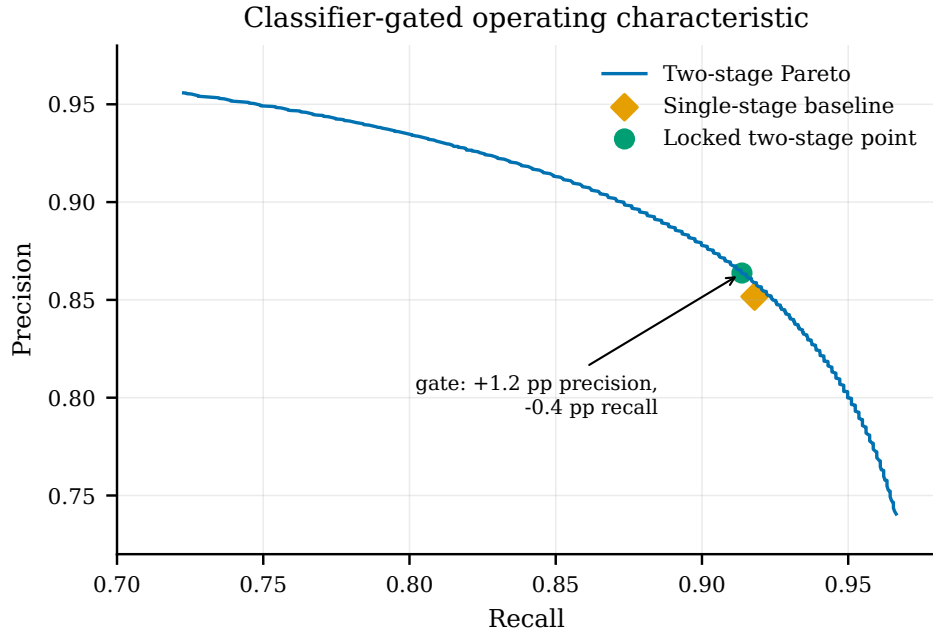


Figure 5.9: Two-stage operating-characteristic Pareto front over the classifier and U-Net thresholds. The annotated point shifts precision +1.2 pp and recall -0.4 pp.

Table 5.4: Five-seed test means for the base and attention-gated U-Nets.

Metric	Base U-Net mean	Attention U-Net mean	Delta
Precision	0.847	0.848	+0.001
Recall	0.923	0.919	-0.005
F_1	0.884	0.882	-0.002
Dice	0.884	0.882	-0.002

5.8 Training and inference sensitivity

5.8.1 Data efficiency

Retraining the winning configuration on nested 30/50/70/100 % subsets of the training images (validation and test held fixed) shows validation Dice flat to 50 %, rising at 70 %, then near-constant (Figure 5.10). Because the hyperparameters are transferred from the 100 %-tuned winner rather than re-tuned per fraction, the curve is a single-seed, indicative learning curve rather than an optimised estimate at each data size. The fixed threshold-0.5 validation Dice values are 0.842, 0.842, 0.854, and 0.856, respectively, so most of the gain occurs before the final fraction. On this single-seed curve the plateau suggests that the 178-image subset does not obviously bind this architecture, which is consistent with the replication reading of Section 5.1.

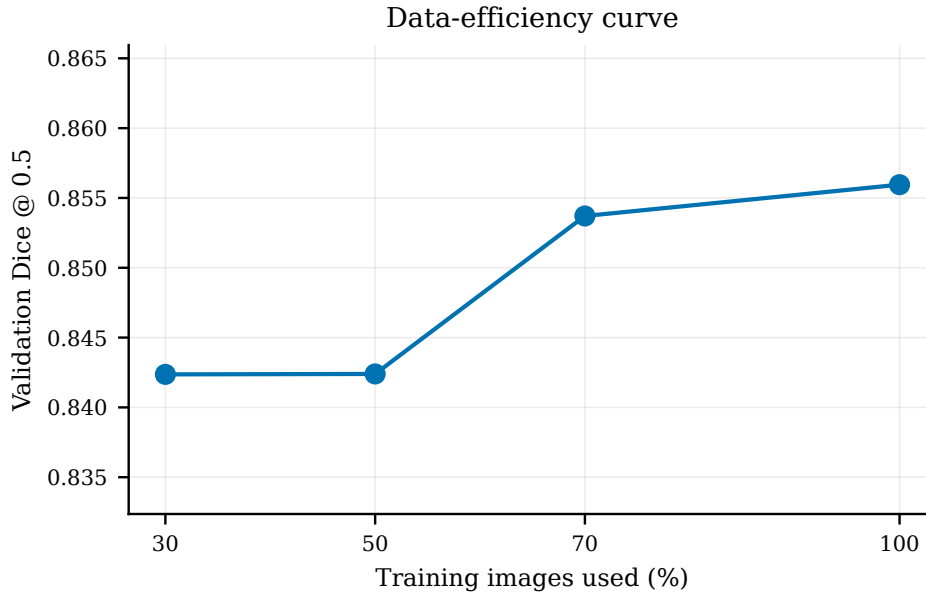


Figure 5.10: Validation Dice versus training-image fraction (fixed-threshold at 0.5). Hyperparameters are transferred from the 100 % winner.

5.8.2 Hard-negative mining

Hard negatives were mined by running the locked model on labelled-empty patches and adding false-positive empty patches back into the training set. Appending 150 mined hard-negatives to the training set gives a small precision gain in a single-seed retrain: +0.007 precision (0.852 $\rightarrow$ 0.859) at essentially unchanged recall (-0.001), for +0.003 Dice. This +0.007 change is within the five-seed precision dispersion (± 0.010 ; Section 5.4), so it is directional rather than resolved; its small size is consistent with the 17.5 % inter-patch false-positive share.

5.8.3 Dilated soft-label pilot

This pilot tests whether the boundary residuals can be reduced by softening the training masks. Instead of training on binary hard labels alone, the trail core remains labelled 1, while a 1 px dilated boundary band is assigned an intermediate target value; validation and test masks remain hard-labelled. The pilot does not improve any overlap, precision, or topology metric (Table 5.5). The soft model predicts wider trails, with its validation-optimal threshold rising from 0.45 to 0.62, giving recall +0.008 but precision -0.022 , exact Dice -0.009 , clDice -0.013 , and a near-identical ± 1 px F_1 (-0.001). The boundary band is absorbed into wider predictions rather than sharpening localisation. This is a null result, consistent with the boundary-placement interpretation.

Table 5.5: Single-seed dilated soft-label pilot compared with the matched hard-label winner. Validation and test masks remain hard-labelled, so the hard-label ± 1 px F_1 is the matched-seed value rather than the five-seed mean in Section 5.6.2.

Metric	Hard-label winner	Dilated-soft pilot	Delta
Exact Dice	0.884	0.875	-0.009
± 1 px F_1	0.963	0.962	-0.001
clDice	0.847	0.835	-0.013

5.8.4 Ensembling with test-time augmentation

This inference-time check asks whether averaging over model seeds and simple test-time transforms improves the locked single-model result without changing training. A five-checkpoint ensemble with test-time augmentation reaches Dice 0.8885 (precision 0.852, recall 0.929 at threshold 0.44), against the single-model 0.8836. The +0.005 Dice improvement is comparable to the five-seed dispersion of the single model, so it is not treated as a material gain. Given the added inference cost, the ensemble is therefore a robustness check rather than a compelling production improvement for this dataset.

5.8.5 Probability-stratified Hough check

This final inference-time check asks whether the Hough stage loses useful confidence information by reducing the U-Net probability map to a binary canvas. Replacing the binary Hough input with a probability-stratified canvas is a clean null: pixel recall changes by +0.00003, the patch-level false-negative rate is identical, and Hough false-positive pixels rise from 2,725,702 to 2,785,274. The result suggests that the Hough limitation is not missing probability weighting: the locked binary stage is already recall-saturated, and probability stratification adds false-positive pixels without measurable recall benefit.

Chapter 6

Discussion

6.1 Deviations from the target paper

Faithful replication is a matter of degree, so Table 6.1 makes the remaining departures from Stoppa et al. [27] explicit. They are forced choices where the paper omits training hyperparameters; deliberate methodology choices such as image-level stratified splits, a validation-swept threshold, and sampled-plus-parity testing; and implementation conventions such as the PyTorch port and logits-with-external sigmoid output. Alignment only in spirit, as with the squared-denominator Dice and stride-528 tiling, is not described as paper-faithful.

The swept threshold and different test population change the headline comparison, chiefly by confounding precision and leaving recall as the closest like-for-like quantity (Section 5.1). The Hough comparison is checked on a full-coverage canvas, but strict precision remains sensitive to the 3px line-rasterisation convention (Section 5.3). The remaining departures were not individually isolated: architecture tests lock the framework port and output convention, image-level splitting removes patch-level leakage, and calibrated noise was validation-screened before adoption.

6.2 Interpreting the headline comparison

Section 6.1 leaves recall as the like-for-like quantity, and even that spans different populations - the 32 test images here against the paper’s 20,000 patches of unreported class balance. For precision the prevalence confound is measured, not assumed: the fall from 0.852 to 0.780 at identical recall and fixed threshold is the direct size of the base-rate effect, while Section 5.6 localises the annotation confound at about one pixel. Evaluating at the paper’s fixed 0.58 rather than the swept 0.45 raises precision by only 0.025, so the threshold is a minor contributor.

The Hough comparison is favourable for completeness but not for strict precision:

Table 6.1: Documented deviations from Stoppa et al. (2024).

Aspect	This work	Target paper
Framework	PyTorch	Keras / TensorFlow
Architecture source	Reconstructed from the released Keras model, reimplemented in PyTorch	Released Keras model
Output convention	Logits; sigmoid in loss/eval	Built-in final sigmoid
Dice formulation	Squared-denominator (released ASTA code)	Standard Dice (paper text)
Patch tiling	528^2 at stride 528 (20×20 grid)	528^2 ; tiling unspecified
Threshold	Validation PR sweep (F_1 -optimal)	Fixed 0.58
Splits	Image-level, trail-pixel-stratified; 122/24/32	Not specified
Test sampling	Sampled 1:3 plus parity (all negatives)	20,000-patch set
Hough input	Patch-reconstructed; full-coverage parity	Dense full image
Hyperparameters	Validation-only Optuna search	Not published
Noise augmentation	Calibrated train-only signal-dependent noise	Not specified

the same gain in pixel completeness (Section 5.3) cuts strict pixel precision from 0.780 to 0.410, about half of which is sensitive to the 3px line-rasterisation convention in a line-thickness sweep rather than necessarily reflecting genuine false detection (Figure B.4). On this full-coverage tiled canvas, verified pixel-for-pixel against the parity canvas, this is comparable completeness, not superiority. The same discipline applies to the tolerant scores: the ± 1 px precision 0.946 locates the residual at the boundary scale but is never set against the paper’s strict 0.94/0.94, and these diagnostics hold across the five seeds (Figure B.5).

6.3 Limitations

The findings above are bounded by seven limitations.

- **Mask under-coverage.** The annotations occasionally omit faint endpoints and low-contrast stretches (Section 2.5). The effect is asymmetric: a correct detection of an unlabelled trail pixel scores as a false positive, and a model trained on such labels also learns to suppress faint extensions. Label-referenced precision is therefore a lower bound where masks omit visible trail pixels. The blinded re-annotation (Section 5.6.6) finds no systematic width difference from the labels; a multi-annotator audit would measure any residual under-coverage directly.

- **Dataset size.** The subset holds 178 images, of which 32 form the test set. Image-level bootstrap intervals are accordingly several times wider than patch-level ones (Section 5.4), the image-level detection rate saturates, and the protocol-sensitivity results (data efficiency, hard negatives, soft labels) are single-seed pilots.
- **Sampled evaluation distribution.** The sampled test set keeps a 1:3 positive-to-negative patch prior, which inflates precision relative to the natural, roughly 93%-empty population; the parity evaluation restores the negatives but is still scored at the sampled-validation threshold. The sampled-to-parity precision drop (0.852 to 0.780) bounds the size of this bias; recall is unaffected because all positive patches are retained.
- **Hough rasterisation convention.** The parity Hough canvas is a verified full-coverage tiled prediction, not a patch-sampling artefact; the one residual is the 3px line-drawing convention, which accounts for about half of the post-Hough precision fall.
- **Single telescope.** All training and quantitative evaluation use MeerLICHT data; transfer to another instrument’s pixel scale, point-spread function, or background is untested, and the DECam application (Figure 5.6) is qualitative only. The DECam montage is therefore a transfer sanity check, not evidence of deployable cross-instrument performance.
- **PNG-only inputs.** The 8-bit renders cap the dynamic range and carry no WCS, exposure, or photometric headers. Trails near the display floor may be undetectable in principle, and the contrast statements of Section 5.6.4 are display-space proxies. The direction of any bias is unclear, because the labels were drawn on the same renders.
- **Single-band imagery.** Each frame provides one display channel, so no claim extends to multi-band trail morphology [36].

6.4 Scientific implications

The error characterisation carries three implications beyond this replication, and a practical reading for deployment.

First, where masks omit visible trail pixels, strict label-referenced precision is best read as a conservative estimate, and the Hough stage makes this concrete: it extrapolates linear extent the network under-detects, but where the annotation also omits that extent the recovered pixels score as false positives - the metric can penalise the recovery the stage was built for. Reported precision for this detector family should therefore be read

alongside a statement of label quality; comparisons across papers with different annotation conventions conflate detector quality with labelling convention.

Second, pixel-level false-negative rates have an instance-level blind spot: the semantic formulation labels every trail pixel identically, so two near-parallel trails merged into one detection lose nothing under any pixel metric here. Freshly launched constellation “trains” produce precisely this geometry, which matters wherever downstream consumers need trail counts rather than masks.

Third, the boundary-placement finding suggests a practical reporting standard. A majority of false-positive pixels lie within about one pixel of the labelled boundary, no tested change resolves that residual, and the blinded re-annotation (Section 5.6.6) is consistent with the reading that strict performance is limited by annotation granularity. Evaluations of thin-artefact detectors should therefore record a mask-width convention and report a ± 1 px boundary-tolerant operating point alongside the strict score, so that boundary convention and detection failure are separated rather than summed.

In practice, the decomposition reads as deployment guidance. Mask-dilation before stacking would absorb a one-pixel boundary residual, so the strict-precision gap may matter less for masking (untested). The classifier gate is best treated as a compute feature - it skips 24.7% of U-Net forwards, more at natural prevalence - with precision headroom capped by the 17.5% inter-patch share. Thresholds do not transfer across prevalence, so a deployed detector should re-select its operating point at the deployment prevalence.

Chapter 7

Conclusions and Future Work

7.1 Conclusions

This dissertation replicated the Stoppa et al. [27] U-Net plus probabilistic Hough satellite-trail detector on a 178-image MeerLICHT subset, with the architecture recovered from the released model and the model and threshold locked on validation data alone. The replication reproduces the paper’s recall closely (five-seed sampled-test recall 0.923 ± 0.007 against 0.94, Dice 0.884 ± 0.004) and its Hough completeness (trail-pixel recall $0.918 \rightarrow 0.990$ on the full-coverage parity canvas). Strict pixel precision sits below the published level (0.847 sampled, 0.780 at parity) and is prevalence-dependent, leaving recall as the like-for-like comparison across test populations.

The follow-up analyses localise the precision gap rather than merely reporting it: a one-pixel boundary tolerance raises five-seed precision from 0.847 to 0.946, and 57% of false positives lie within one pixel of a labelled trail. A blinded single-author re-annotation then tests the main ambiguity: the original labels, the re-annotation, and the model all have median width 6 px, with no systematic width bias and no statistically resolved strict excess over the original-label comparison under this design. The tested interventions - classifier gate, attention U-Net, data efficiency, hard-negative mining, soft labels, ensembling, and probability-stratified Hough - are small or null under their stated protocols. The null variants still matter: they close off obvious capacity and protocol explanations without making the boundary diagnosis decisive. None resolves the residual materially, so boundary-placement remains the most consistent reading.

The project also demonstrates a methodology. Faithful replication - architecture recovered from released weights, preregistered selection, and sampled-versus-parity evaluation kept distinct - surfaced at each step a discrepancy a looser reproduction would have absorbed into an unexplained shortfall. Treating the remaining gap as an object of analysis converted a headline deficit into a measured, bounded mechanism. Conducted at this fidelity, replication is not a re-implementation exercise but an instrument for locating

where a published result’s accuracy resides: in the detector, in the labels, or in the metric.

7.2 Future work

Several directions follow. The strongest remaining check is a *multi-annotator gold-standard audit*: independent annotators would establish inter-annotator agreement and convert the single-author between-source estimate (Section 5.6.6) into an agreement-bounded estimate of how much of the strict precision gap is mask granularity rather than detector error. A smaller label-policy ablation would test component-level mask cleaning, such as removing isolated small objects before training and evaluation, while treating the cleaned masks as a new target rather than a correction to the locked results. A second priority is a *labelled cross-instrument evaluation* - the DECam application here is qualitative only (Figure 5.6) - with a classical baseline in the spirit of Serrano Mendoza et al. [24], on which attention [21] and richer architectures could be re-examined where the matched-capacity null may not persist.

Secondary directions follow from the metric’s instance-level blindness. An *instance-level* detection score over matched trail instances would expose the merged near-parallel-trail failure mode that pixel rates cannot register, made quantitative by a synthetic constellation-“train” probe. Trail *removal* - combining the detected mask with the fitted Hough geometry to interpolate across the trail - is the natural downstream step, though on 8-bit renders this is cosmetic reconstruction rather than photometric recovery.

Bibliography

- [1] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama. Optuna: A next-generation hyperparameter optimization framework. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 2623–2631, 2019. doi: 10.1145/3292500.3330701.
- [2] C. G. Bassa, O. R. Hainaut, and D. Galadí-Enríquez. Analytical simulations of the effect of satellite constellations on optical and near-infrared observations. *Astronomy & Astrophysics*, 657:A75, 2022. doi: 10.1051/0004-6361/202142101. arXiv:2108.12335.
- [3] D. Bektešević and D. Vinković. Linear feature detection algorithm for astronomical surveys – I. Algorithm description. *Monthly Notices of the Royal Astronomical Society*, 471(3):2626–2641, 2017. doi: 10.1093/mnras/stx1565.
- [4] S. Bloemen, P. Groot, P. Woudt, et al. MeerLICHT and BlackGEM: custom-built telescopes to detect faint optical transients. In *Ground-based and Airborne Telescopes VI, Proceedings of the SPIE*, volume 9906, page 990664, 2016. doi: 10.1117/12.2232522.
- [5] J. Carrillo Navarro et al. StreakMind: AI detection and analysis of satellite streaks in astronomical images with automated database integration. *Astronomy & Astrophysics*, 708:A211, 2026. doi: 10.1051/0004-6361/202558754. arXiv:2605.03429.
- [6] CelesTrak (T. S. Kelso). SATCAT and active-satellite catalogue. <https://celestrak.org/satcat/>, 2026. Active-satellite element table parsed 2026-05-27.
- [7] X. Chen, H. Gu, C. Wang, H. Mu, J. Zheng, J. Du, J. Ren, Z. Fan, and J. Li. Artificial satellite trails detection using U-Net deep neural network and line segment detector algorithm. *Publications of the Astronomical Society of the Pacific*, 137(9):094504, 2025. doi: 10.1088/1538-3873/ae051f. arXiv:2509.16771.
- [8] R. O. Duda and P. E. Hart. Use of the Hough transformation to detect lines and curves in pictures. *Communications of the ACM*, 15(1):11–15, 1972. doi: 10.1145/361237.361242.
- [9] Federal Communications Commission, Space Bureau. Order DA 26-36: SpaceX Gen2 Starlink authorisation. <https://docs.fcc.gov/public/attachments/DA-26-36A1.pdf>, 2026.

- [10] O. R. Hainaut and A. P. Williams. Impact of satellite constellations on astronomical observations with ESO telescopes in the visible and infrared domains. *Astronomy & Astrophysics*, 636:A121, 2020. doi: 10.1051/0004-6361/202037501. arXiv:2003.01992.
- [11] K. He, X. Zhang, S. Ren, and J. Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015. doi: 10.1109/ICCV.2015.123.
- [12] Z. Hosenie, S. Bloemen, P. Groot, R. Lyon, B. Scheers, B. Stappers, F. Stoppa, P. Vreeswijk, S. De Wet, M. Klein Wolt, R. Le Poole, V. McBride, K. Paterson, D. L. A. Pieterse, and P. Woudt. MeerCRAB: MeerLICHT classification of real and bogus transients using deep learning. *Experimental Astronomy*, 51(2):319–344, 2021. doi: 10.1007/s10686-021-09757-1.
- [13] P. V. C. Hough. Method and means for recognising complex patterns. *US Patent 3069654*, 1962.
- [14] D. Kalamkar, D. Mudigere, N. Mellempudi, et al. A study of BFLOAT16 for deep learning training. *arXiv preprint*, 2019. arXiv:1905.12322.
- [15] S. Kruk et al. The impact of satellite trails on Hubble Space Telescope observations. *Nature Astronomy*, 7:262–268, 2023. doi: 10.1038/s41550-023-01903-3.
- [16] J. Matas, C. Galambos, and J. Kittler. Robust detection of lines using the progressive probabilistic Hough transform. *Computer Vision and Image Understanding*, 78(1): 119–137, 2000. doi: 10.1006/cviu.1999.0831.
- [17] P. Micikevicius, S. Narang, J. Alben, et al. Mixed precision training. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1710.03740.
- [18] F. Milletari, N. Navab, and S.-A. Ahmadi. V-Net: Fully convolutional neural networks for volumetric medical image segmentation. In *Fourth International Conference on 3D Vision (3DV)*, pages 565–571, 2016. doi: 10.1109/3DV.2016.79.
- [19] P. Mróz et al. Impact of the SpaceX Starlink satellites on the Zwicky Transient Facility survey observations. *The Astrophysical Journal Letters*, 924(2):L30, 2022. doi: 10.3847/2041-8213/ac470a. arXiv:2201.05343.
- [20] G. Nir, B. Zackay, and E. O. Ofek. Optimal and efficient streak detection in astronomical images. *The Astronomical Journal*, 156(5):229, 2018. doi: 10.3847/1538-3881/aaddff. arXiv:1806.04204.
- [21] O. Oktay et al. Attention U-Net: Learning where to look for the pancreas. In *Medical Imaging with Deep Learning (MIDL)*, 2018. arXiv:1804.03999.

- [22] M. Paillassa, E. Bertin, and H. Bouy. MaxiMask and MaxiTrack: two new tools for identifying contaminants in astronomical images using convolutional neural networks. *Astronomy & Astrophysics*, 634:A48, 2020. doi: 10.1051/0004-6361/201936345. arXiv:1907.08298.
- [23] O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241. Springer, 2015. doi: 10.1007/978-3-319-24574-4_28.
- [24] A. Serrano Mendoza, M. Rawls, and A. A. Plazas Malagón. Identifying and measuring satellite streaks in DECam images. *arXiv preprint*, 2026. doi: 10.48550/arXiv.2603.10790. arXiv:2603.10790.
- [25] S. Shit, J. C. Paetzold, A. Sekuboyina, I. Ezhov, A. Unger, A. Zhylyka, J. P. W. Pluim, U. Bauer, and B. H. Menze. clDice – a novel topology-preserving loss function for tubular structure segmentation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021. arXiv:2003.07311.
- [26] D. V. Stark, N. A. Grogan, J. E. Ryon, and R. A. Lucas. Improved identification of satellite trails in ACS/WFC imaging using a modified Radon transform. *arXiv e-prints*, 2026. arXiv:2602.17816.
- [27] F. Stoppa et al. Automated detection of satellite trails in ground-based observations using U-Net and Hough transform. *Astronomy & Astrophysics*, 692:A199, 2024. doi: 10.1051/0004-6361/202451663. arXiv:2407.19461.
- [28] A. J. Storkey, N. C. Hambly, C. K. I. Williams, and R. G. Mann. Cleaning sky survey databases using Hough transform and renewal string approaches. *Monthly Notices of the Royal Astronomical Society*, 347(1):36–51, 2004. doi: 10.1111/j.1365-2966.2004.07211.x.
- [29] S. A. Taghanaki, Y. Zheng, S. K. Zhou, et al. Combo loss: handling input and output imbalance in multi-organ segmentation. *Computerized Medical Imaging and Graphics*, 75:24–33, 2019. doi: 10.1016/j.compmedimag.2019.04.005. arXiv:1805.02798.
- [30] J. A. Tyson et al. Mitigation of LEO satellite brightness and trail effects on the Rubin Observatory LSST. *The Astronomical Journal*, 160:226, 2020. doi: 10.3847/1538-3881/abba3e. arXiv:2006.12417.
- [31] Vera C. Rubin Observatory. Frequently asked questions: impacts from artificial satellites and debris. <https://rubinobservatory.org/for-scientists/frequently-asked-questions/leo-sats>, 2026. Accessed 2026-05-27.

- [32] J. Virtanen, J. Poikonen, T. Säätti, et al. Streak detection and analysis pipeline for space-debris optical images. *Advances in Space Research*, 57(8):1607–1623, 2016. doi: 10.1016/j.asr.2015.09.024.
- [33] C. Walker, J. Hall, et al. SATCON1 report: impact of satellite constellations on optical astronomy and recommendations toward mitigations. Bulletin of the AAS, 52(2); NSF NOIRLab / AAS, 2020.
- [34] C. Walker et al. SATCON2 working group reports. Bulletin of the AAS; NSF NOIRLab / AAS, 2021.
- [35] C. Xu, C. McCully, B. Dong, D. A. Howell, and P. Sen. Cosmic-CoNN: A cosmic-ray detection deep-learning framework, data set, and toolkit. *The Astrophysical Journal*, 942(2):73, 2023. doi: 10.3847/1538-4357/ac9d91. arXiv:2106.14922.
- [36] H.-J. Yu, J.-L. Zheng, and Y. Fang. Using deep learning to identify artificial satellite trails in multi-band photometric astronomical images. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2509.04081. arXiv:2509.04081.
- [37] K. Zhang and J. S. Bloom. deepCR: Cosmic ray rejection with deep learning. *The Astrophysical Journal*, 889(1):24, 2020. doi: 10.3847/1538-4357/ab3fa6. arXiv:1907.09500.

Appendix A

Hyperparameter Search Space

Table A.1 records the Optuna search space; all other training choices are fixed by the methodology in Chapter 4.

Table A.1: Optuna hyperparameter search space.

Parameter	Type	Range / values	Notes
<code>learning_rate</code>	log-uniform	$[10^{-4}, 10^{-3}]$	Common range for all batch sizes
<code>bce_weight</code>	uniform	$[0.2, 0.8]$	$\lambda_{\text{Dice}} = 1 - \lambda_{\text{BCE}}$
<code>batch_size</code>	fixed allocation	$\{8, 16, 32\}$	45 trials total; 15 per batch size

Appendix B

Supplementary Results

B.1 Post-hoc Trail and Error Diagnostics

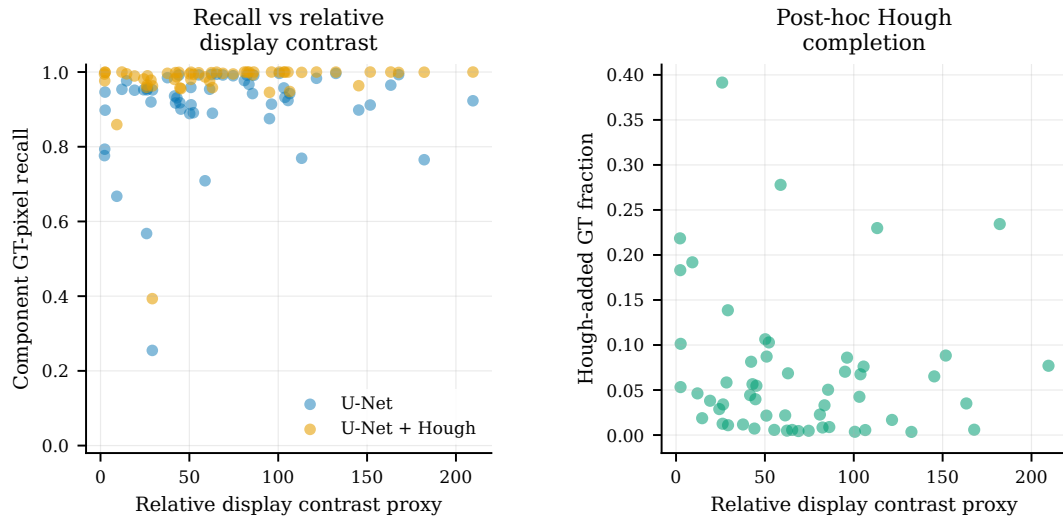


Figure B.1: Relative display contrast versus U-Net recall and Hough completion for the locked winner; the contrast proxy is descriptive, not calibrated flux or SNR.

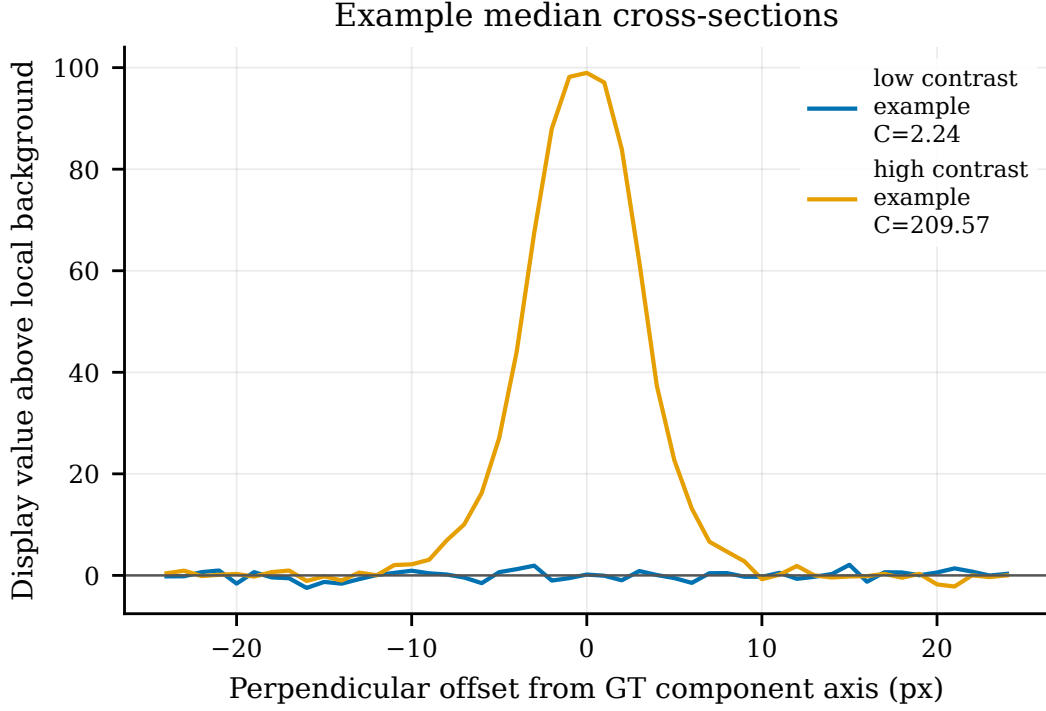


Figure B.2: Median cross-sections through labelled trails in display-space PNG values; descriptive examples of the relative-contrast proxy.

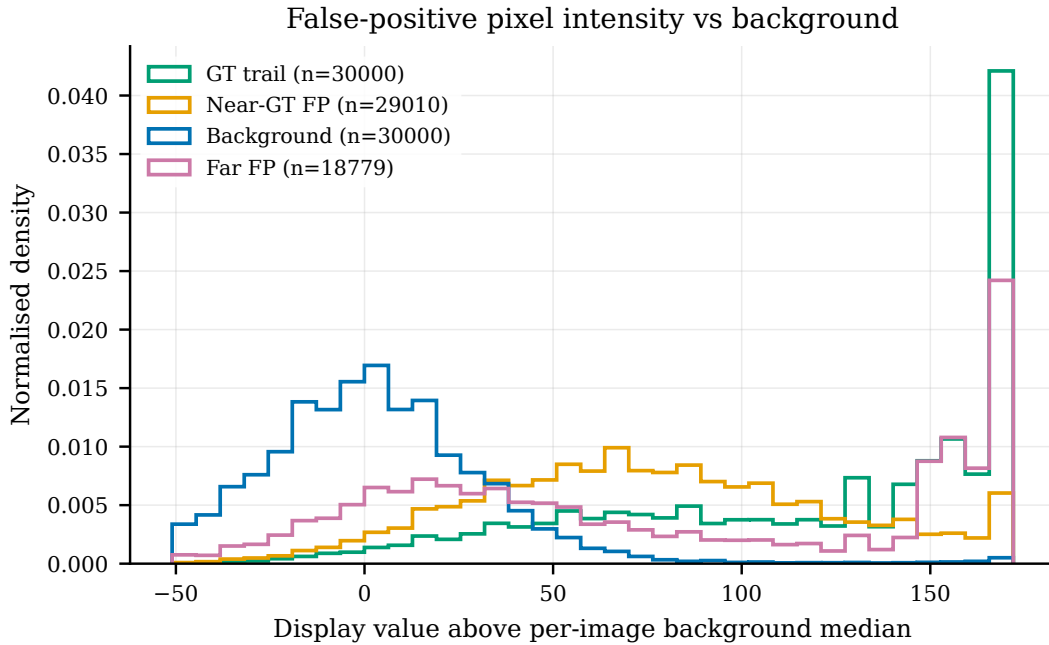


Figure B.3: False-positive pixel display intensity versus labelled trail and background pixels. The KS distances are consistent with mask under-coverage and background confusion, but remain descriptive.

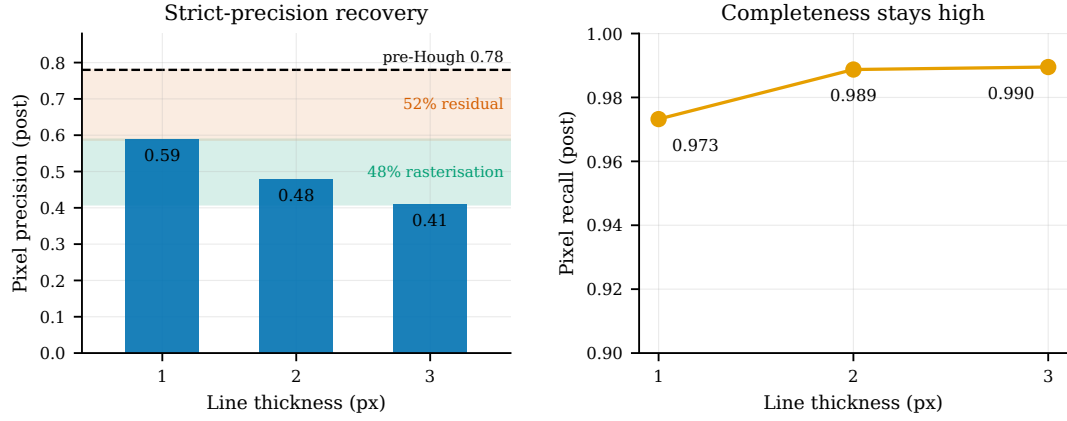


Figure B.4: Post-Hough strict pixel precision (left) and completeness (right) on the parity canvas as the drawn Hough line thickness varies over 1–3 px. Thinning the line from 3 to 1 px recovers roughly half of the $0.780 \rightarrow 0.410$ precision fall while completeness stays above 0.97, consistent with about half the fall arising from the 3 px rasterisation convention over the roughly 6 px masks and the remainder not being recovered by 1 px line thinning.

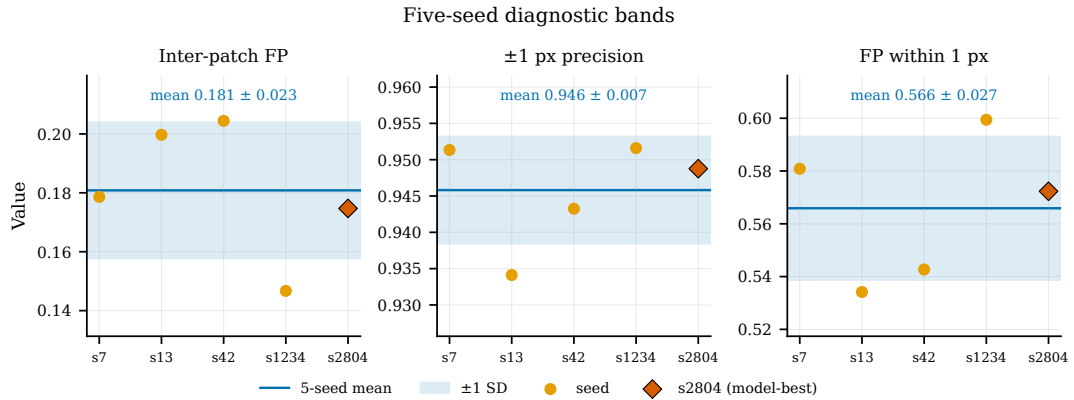


Figure B.5: The three load-bearing boundary diagnostics across the five retraining seeds: inter-patch false-positive share, ± 1 px precision, and false positives within 1 px. Markers are per-seed values; the line and shaded band give the five-seed mean ± 1 SD; the diamond marks the locked winner (seed 2804, model-best), which lies inside every band.

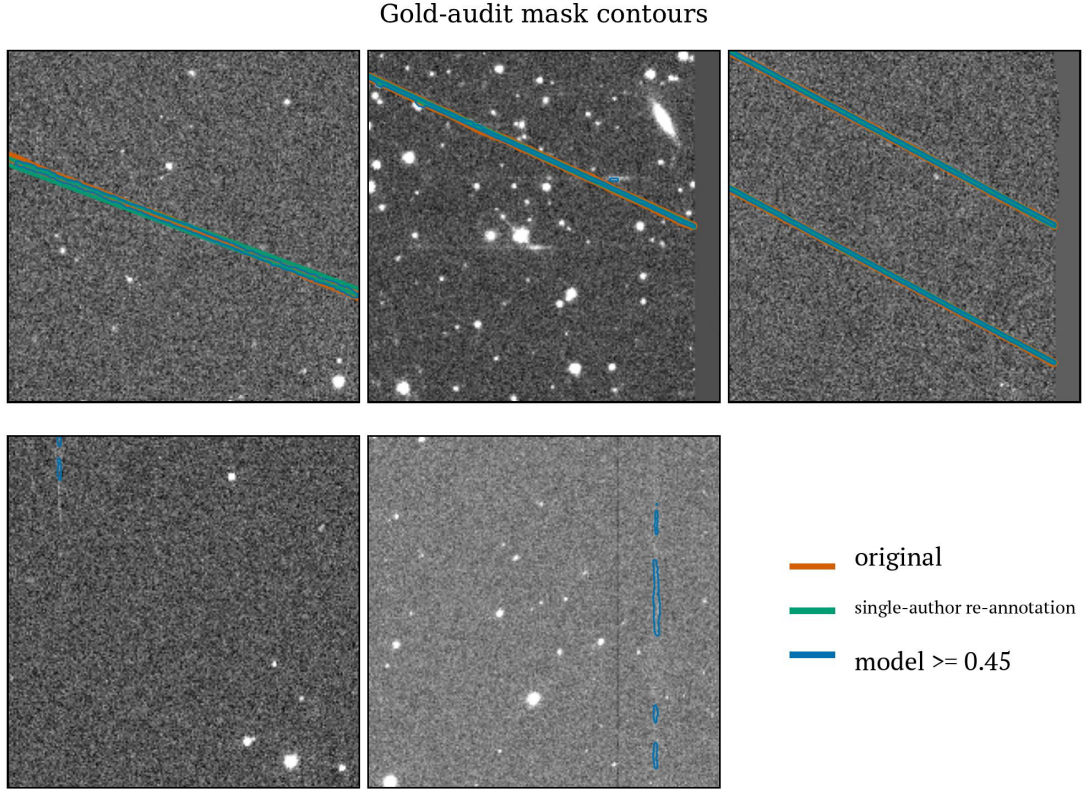


Figure B.6: Blinded re-annotation overlays for five audit crops. Contours show the original MeerLICHT mask (orange), the blinded single-author re-annotation (green), and the locked model prediction at threshold 0.45 (blue). The fixed selection shows the three lowest-index non-empty interior/endpoint crops, where boundaries are near-coincident, and the two lowest-index false-positive-prone controls with model-only over-firing. The single-author re-annotation is not an independent gold standard.

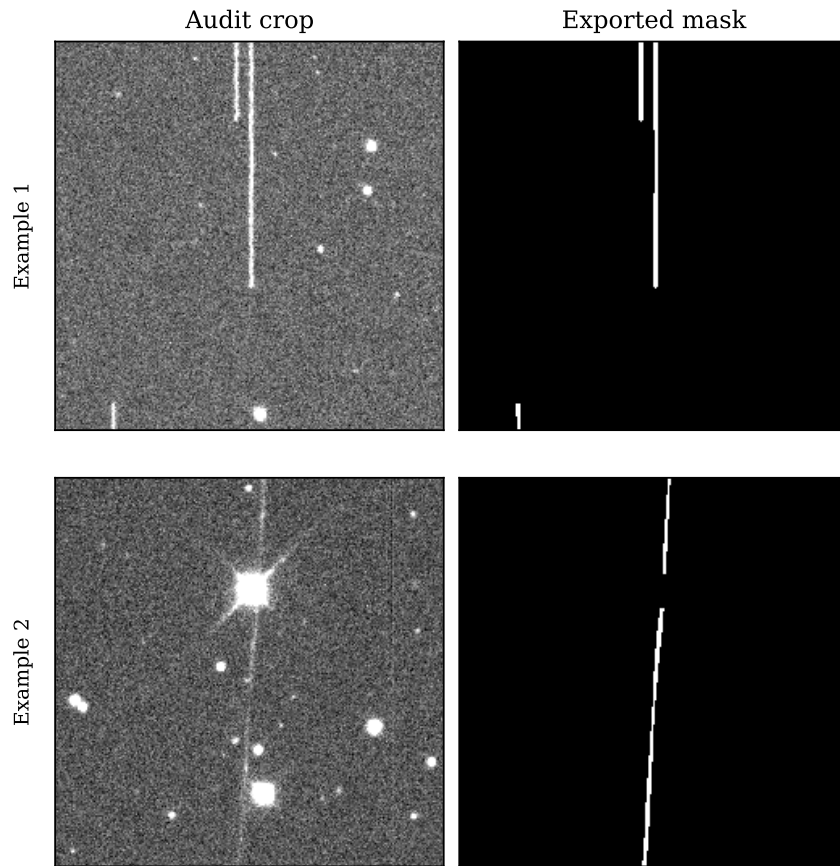


Figure B.7: Illustrative crop–mask pairs from the blinded re-annotation audit. Left: sealed audit crops before annotation; right: exported binary re-annotation masks. The full private gold set is not redistributed. Re-annotations were drawn in GIMP at fixed 400% zoom with no per-crop contrast adjustment, using the Pencil tool with a 1 px hard edge, 100% hardness, 100% opacity, and dynamics off, then exported as binary 0/255 PNG masks.

B.2 Prediction Galleries

FP / TP patch gallery: green = GT, magenta = prediction (overlap = magenta inside green)

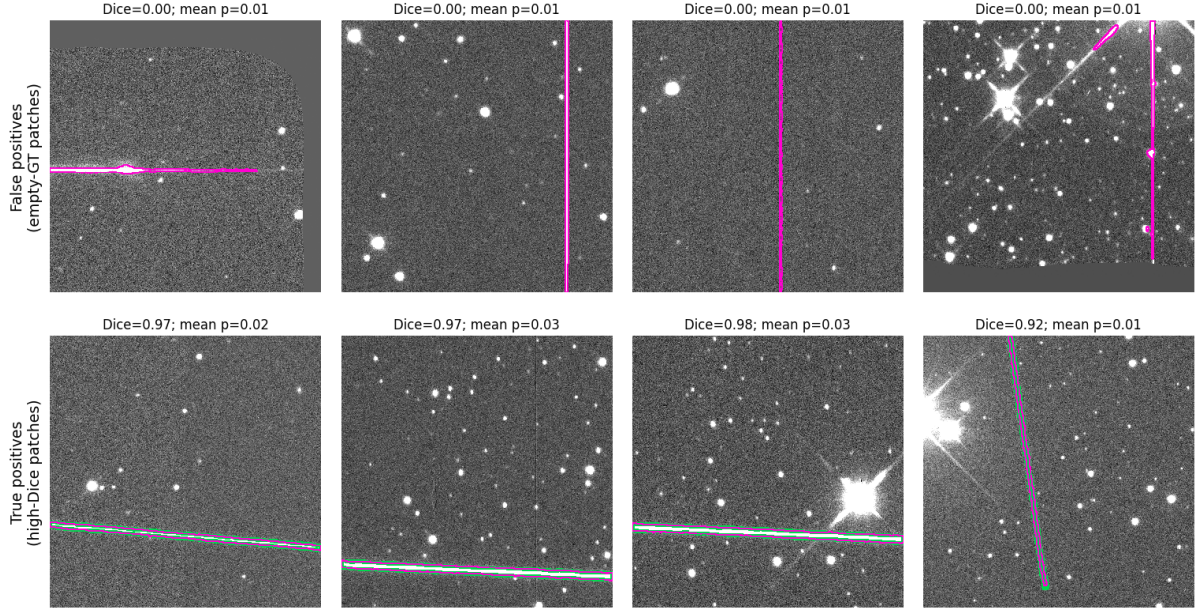


Figure B.8: False-positive and true-positive gallery for the locked winner, including bright-star diffraction-spike examples alongside high-Dice cases where bright stars are correctly ignored.

FP / TP patch gallery: green = GT, magenta = prediction (overlap = magenta inside green)

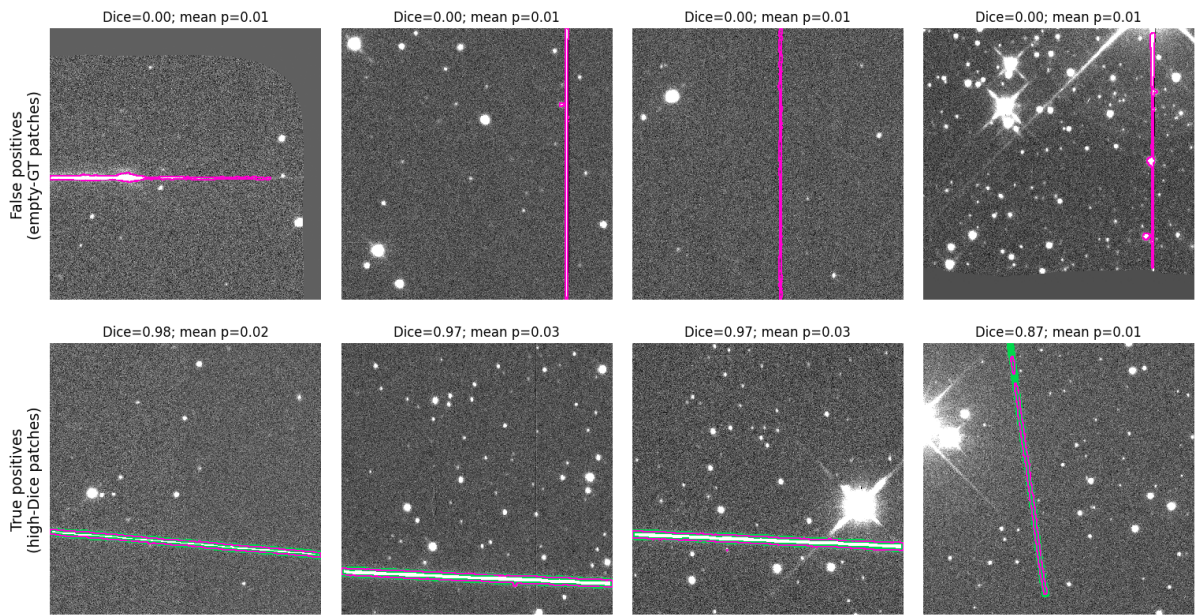


Figure B.9: The same eight patches for the attention-gated U-Net, in the order of Figure B.8, each with that model's own segmentation and Dice score.

Appendix C

Use of Auto-generation Tools

I have used Codex (ChatGPT) and Claude Code to support me across this work, primarily in the following areas:

- Providing advice and usage support for libraries like PyTorch and PIL.
- Generating boilerplate code for plotting, testing and scripting.
- Generating function docstrings.
- Reviewing and auditing project structure for completeness, consistency, accuracy and best practices in software engineering.